

project1: 驱动 OLED

概述

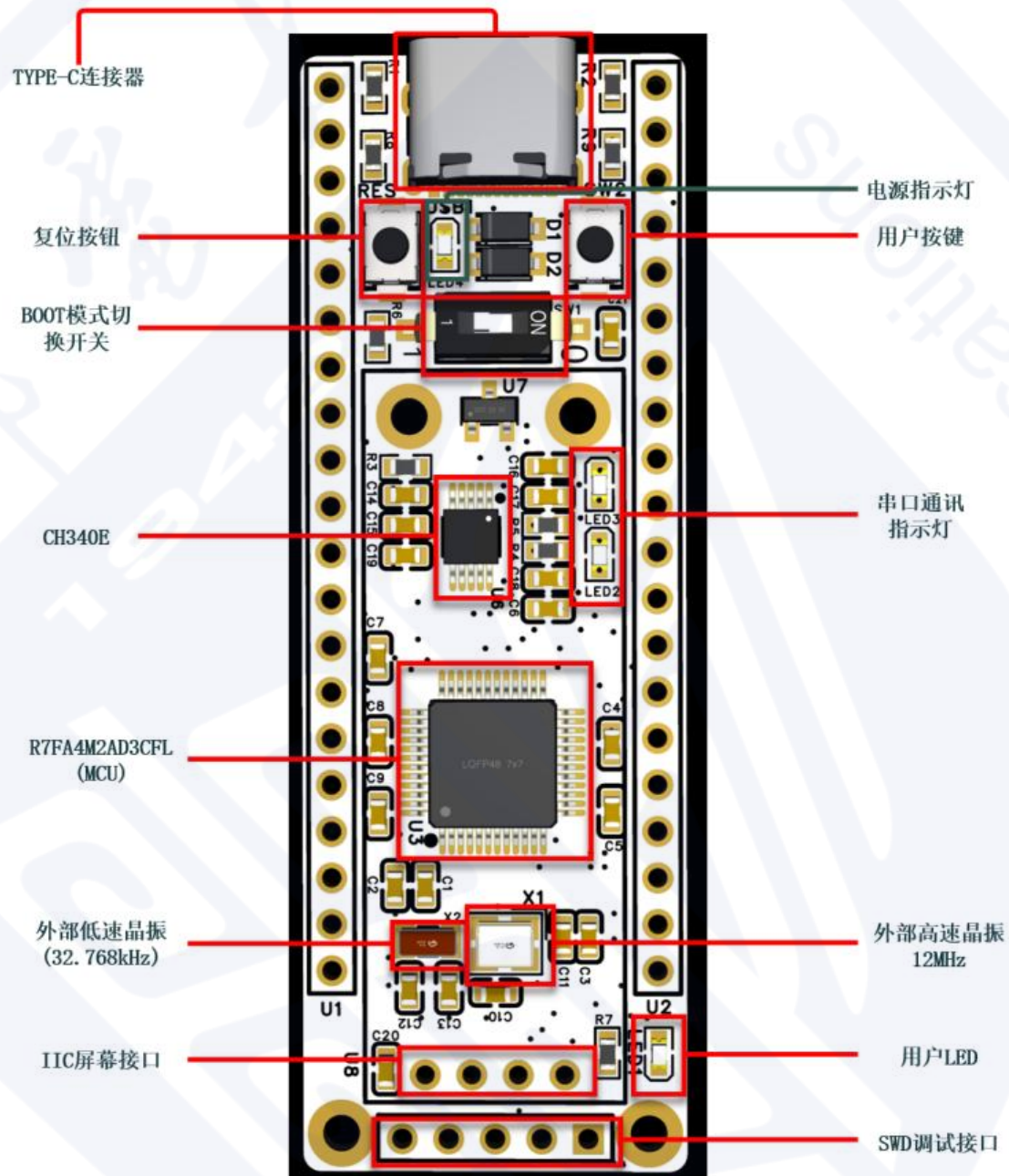
本章旨在通过 I²C 接口驱动 OLED 显示屏（常见型号如 SSD1312），实现图形和文本的显示功能。OLED 屏幕具有低功耗、自发光、对比度高等优点，适用于嵌入式人机界面显示。



硬件准备

首先需要准备一个开发板，这里我准备的是自己绘制的开发板，需要的可以进行申请。

主控为 R7FA4M2AD3CFL

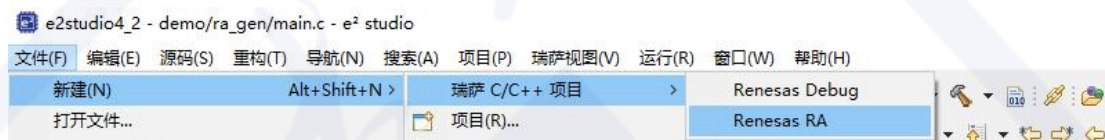


参考程序

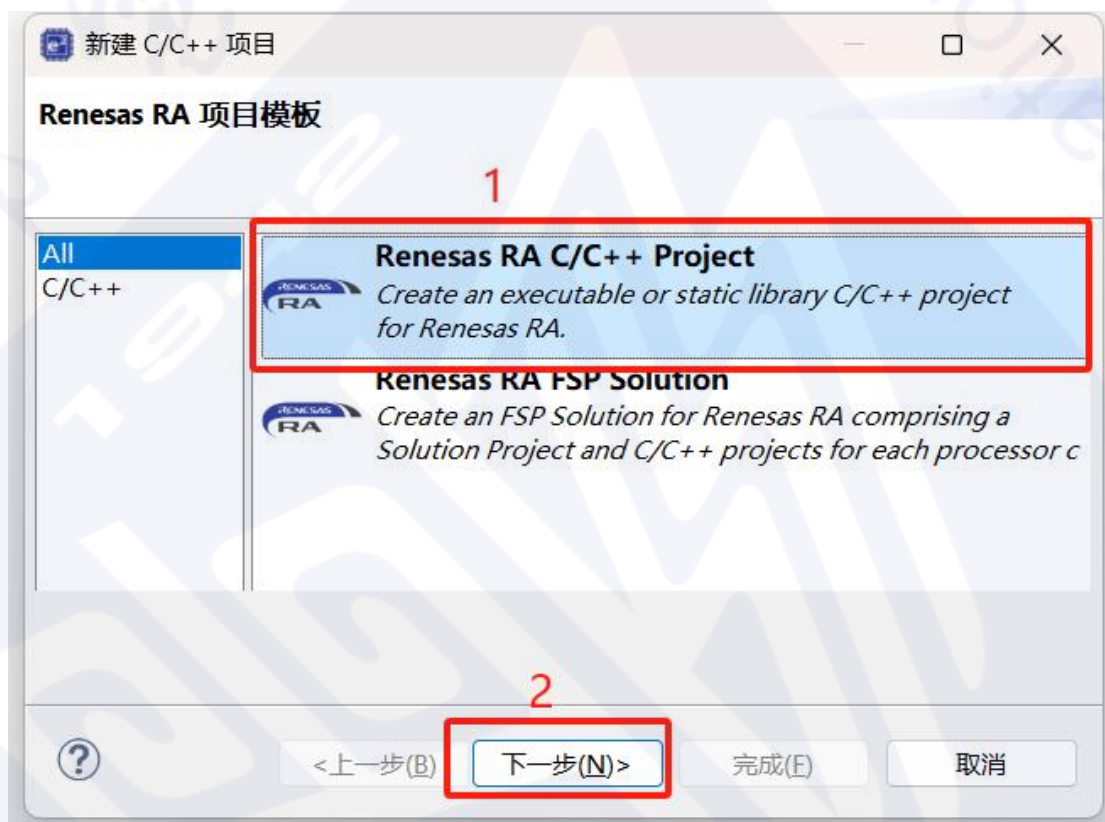
https://github.com/CoreMaker-lab/RA4M2_MINI_OLED

https://gitee.com/CoreMaker/RA4M2_MINI_OLED

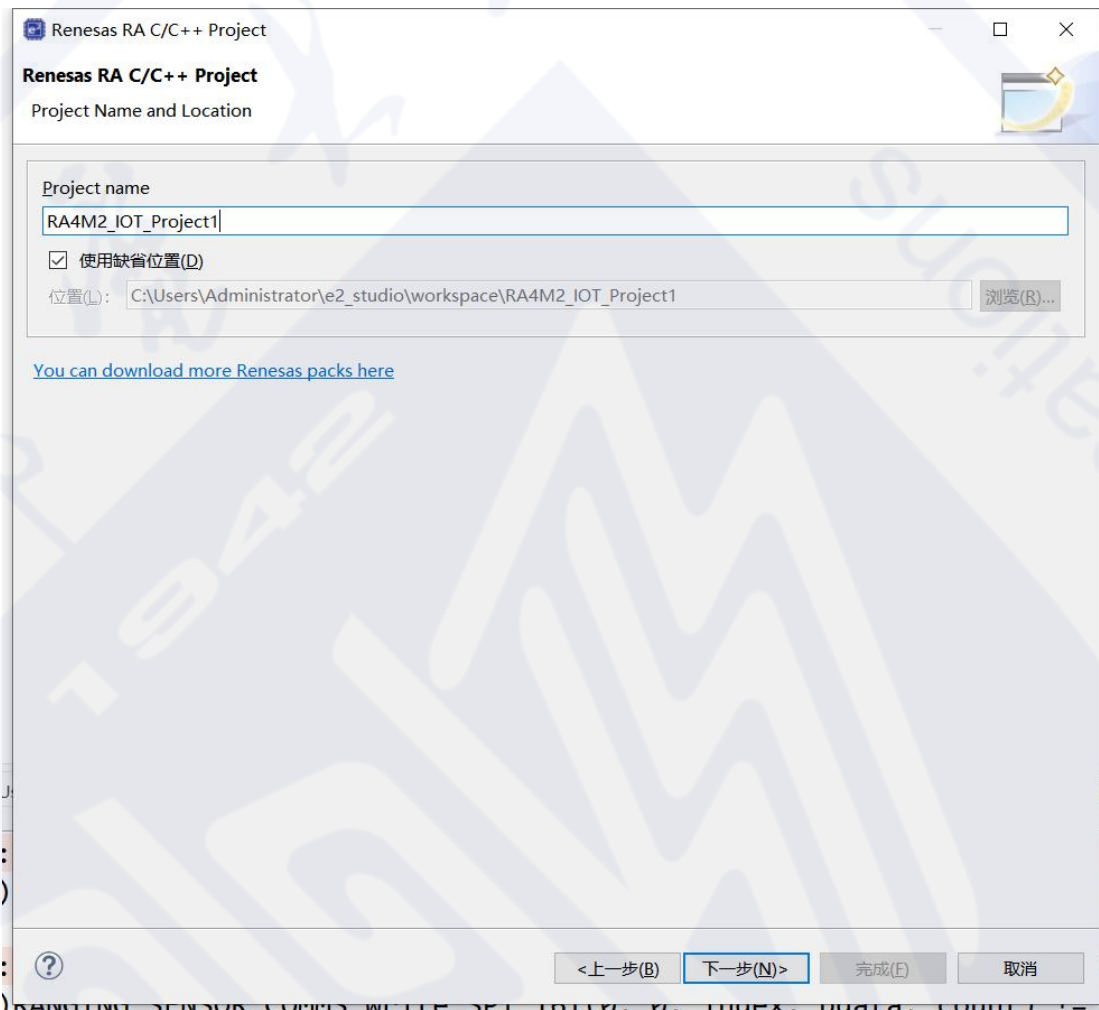
新建工程



工程模板



保存工程路径



芯片配置

本文中使用 R7FA4M2AD3CFL 来进行演示。

Renesas RA C/C++ Project

Renesas RA C/C++ Project

Device and Tools Selection

Device Selection

FSP Version: 5.9.0 **版本**

Board: Custom User Board (Any Device) ...

Device: R7FA4M2AD3CFL **芯片型号**

Core: CM33

Language: ☒ C ☐ C++

Board Description

Device Details

TrustZone	Yes
Pins	48
Processor	Cortex-M33

IDE Project Type

e2 studio managed build

Toolchains

LLVM Embedded Toolchain for Arm

GNU ARM Embedded **工具链**

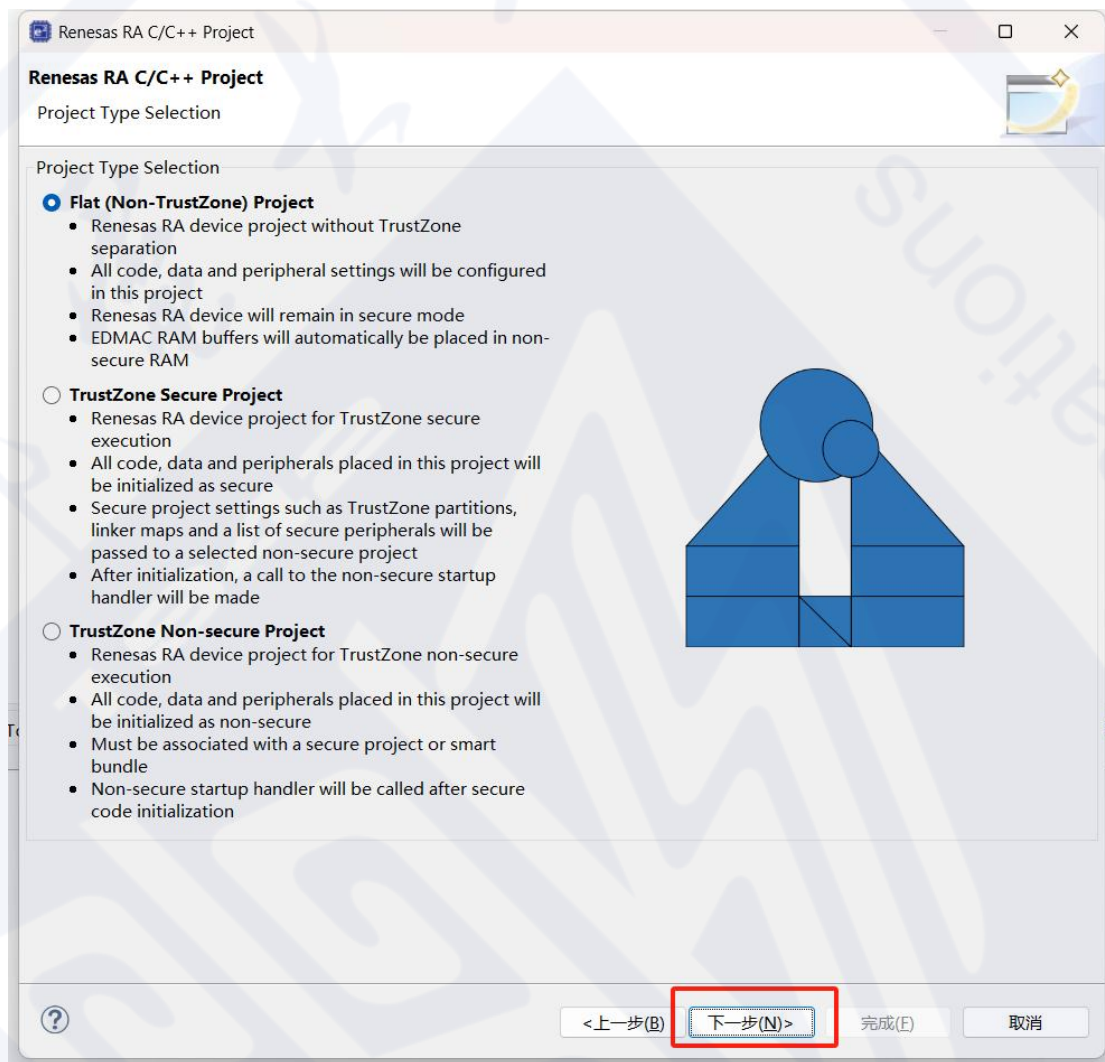
13.2.1.arm-13-7 [Manage Toolchains...](#)

Debugger

J-Link ARM **使用JLink进行调试**

[?](#) <上一步(B) 下一步(N)> 完成(F) 取消

工程模板选择



Renesas RA C/C++ Project

Renesas RA C/C++ Project

Preceding Project or Smart Bundle Selection

☒ None

Choose this option when creating a project for the primary processor core (no preceding project or Smart Bundle).

☐ Preceding Project:

Choose this option if you have access to the preceding bootloader project or TrustZone secure project source code. A debug configuration for Secure Software Debug (SSD) state will be generated where applicable and both secure and non-secure images will be downloaded to the target device. Should you wish to test Non-Secure Software Debug (NSECSO) state, use the Renesas Device Partition Manager to change the Device Lifecycle state as needed.

☐ Smart Bundle:

Resolved location:

Workspace...

File System...

Variables...

Choose this option if you only have access to a pre-programmed device containing TrustZone secure code. The Smart Bundle file (*.sbd) should be obtained from the TrustZone Secure Project developer. A debug configuration for Non-Secure Software Debug (NSECSO) state will be generated and only the non-secure code image will be downloaded to the target device.

Preceding Project/Smart Bundle Details

FSP version

Toolchain

Toolchain version

Board

Device

Core

Zones

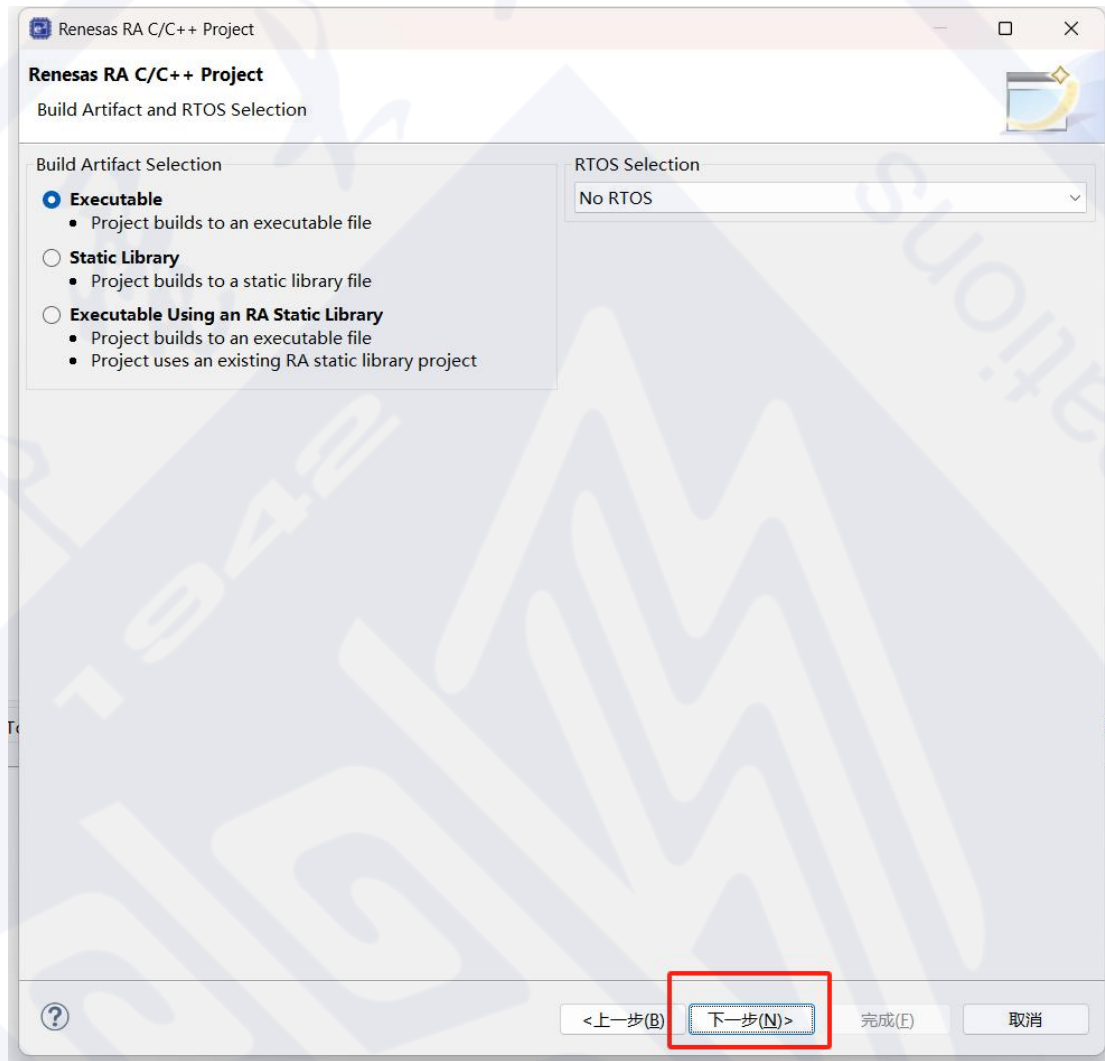
?

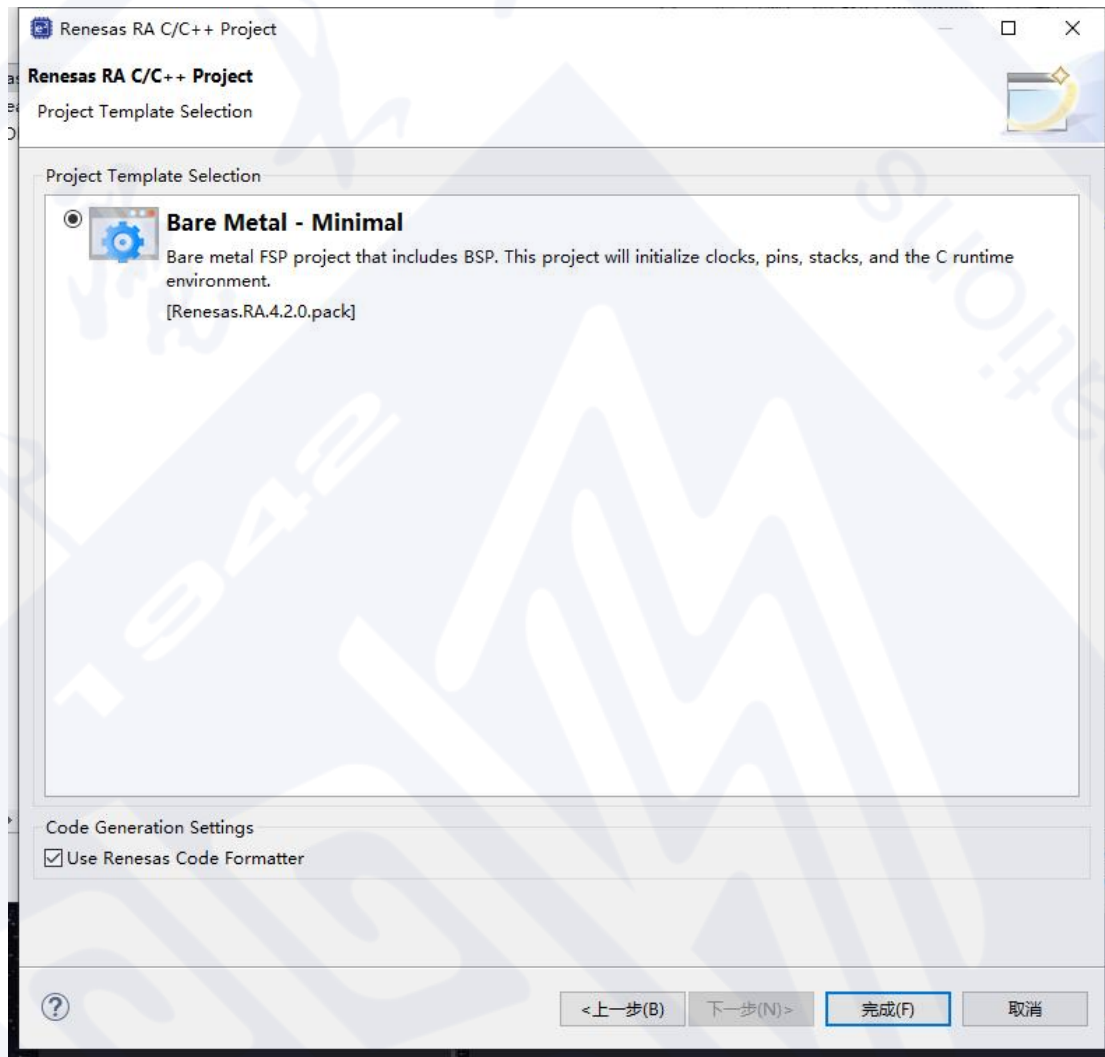
< 上一步(B)

下一步(N) >

完成(E)

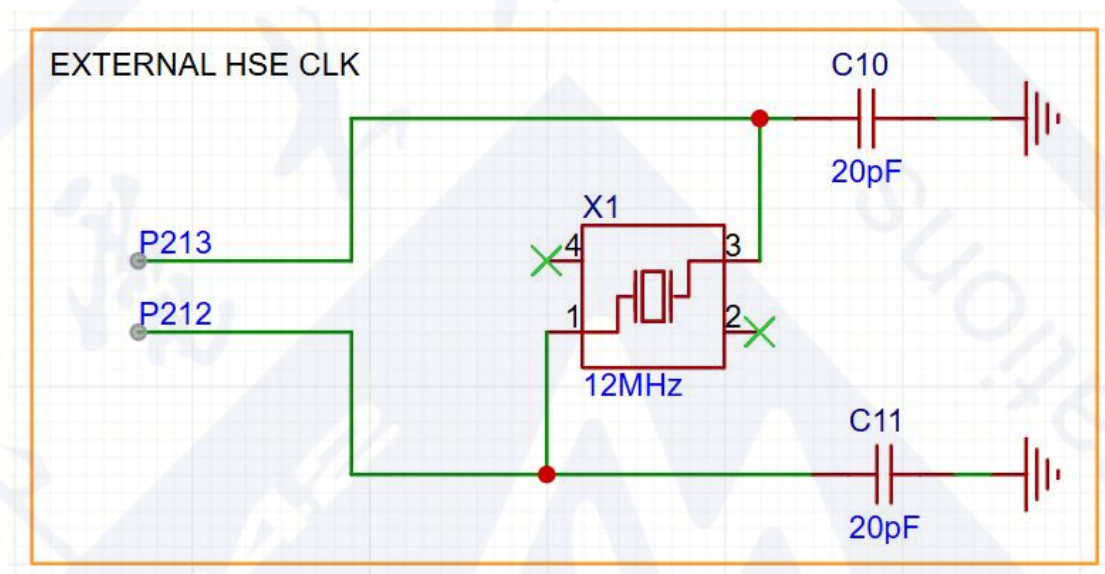
取消



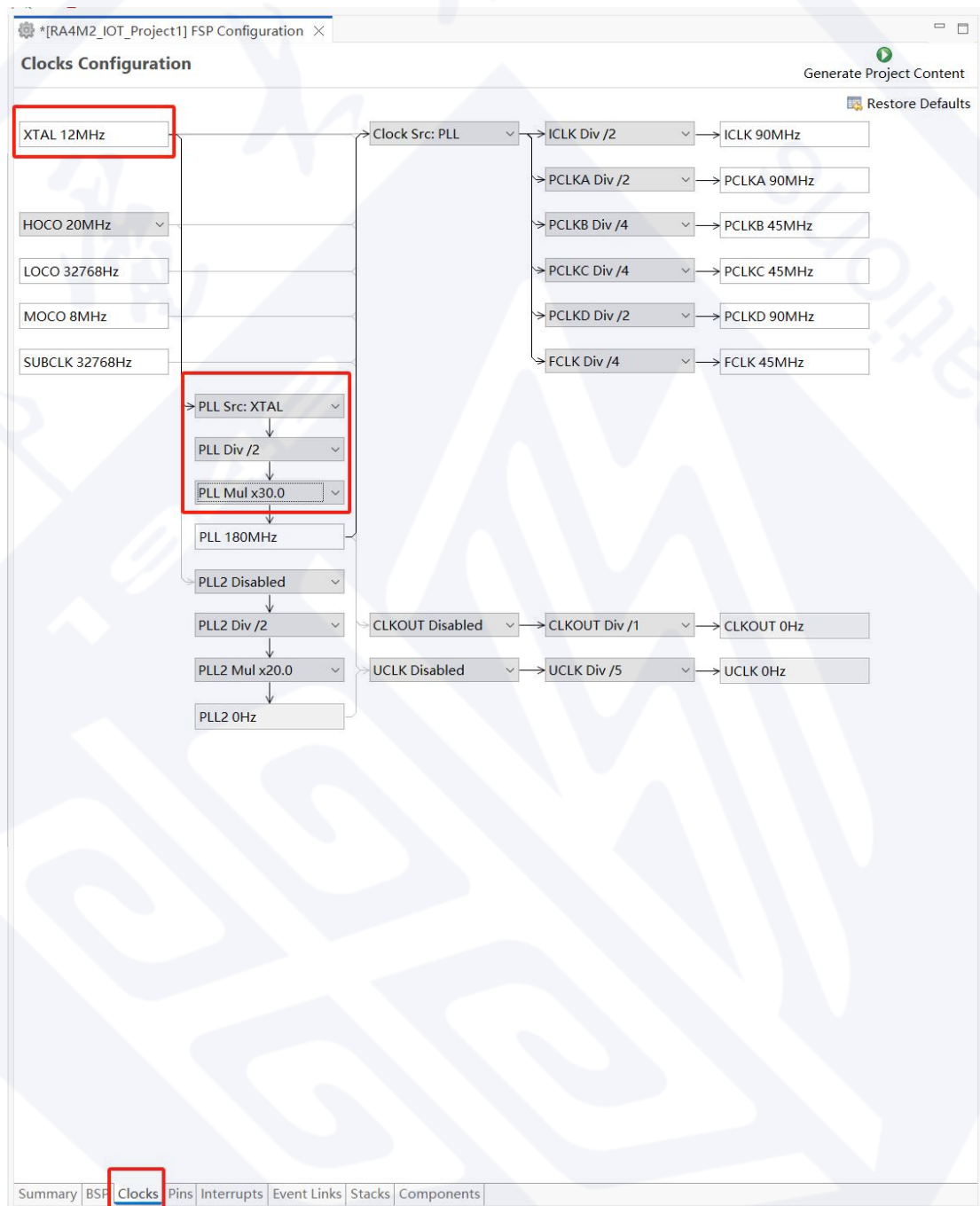


时钟设置

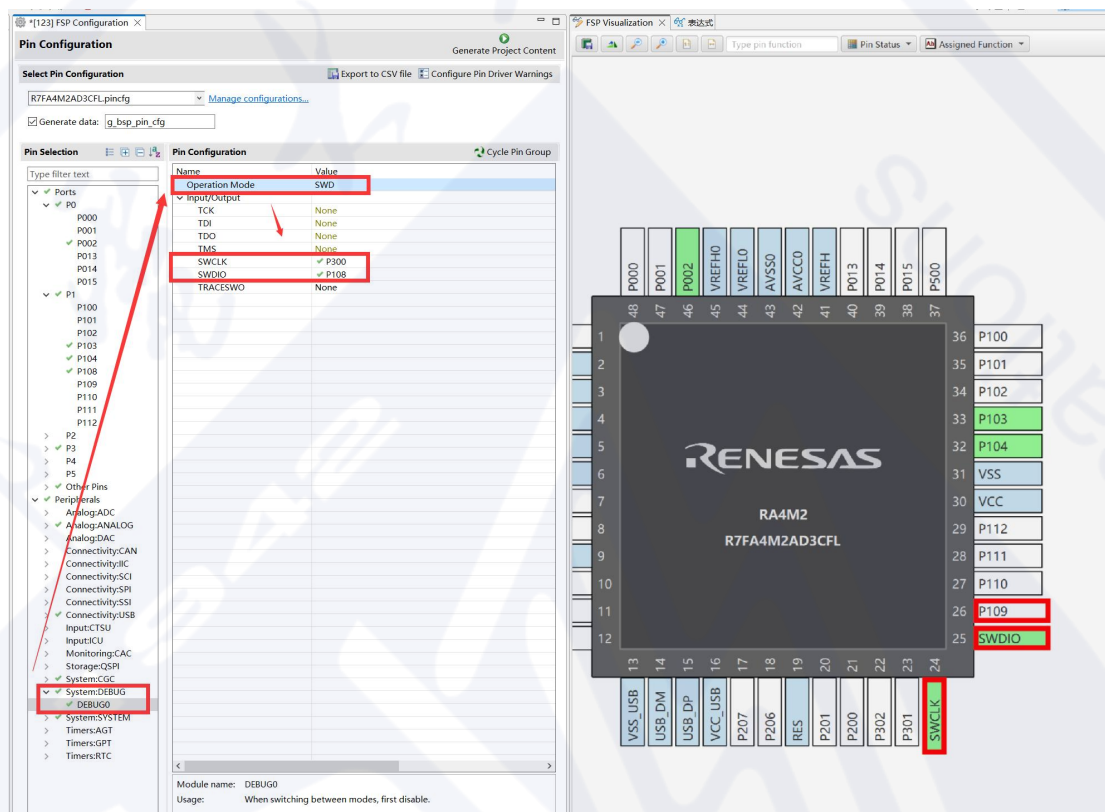
开发板上的外部高速晶振为 12M.



需要修改 XTAL 为 12M。



SWD 调试口设置



OLED 属性配置

查看手册，可以得知 OLED 的 IIC 地址为“b0111100” or “b0111101”，即 0x3c 或者 0x3d。

6.1.5 MCU I²C Interface

The I²C communication interface consists of slave address bit SA0, I²C-bus data signal SDA (SDA_{OUT}/D₂ for output and SDA_{IN}/D₁ for input) and I²C-bus clock signal SCL (D₀). Both the data and clock signals must be connected to pull-up resistors. RES# is used for the initialization of device.

- a) Slave address bit (SA0)
SSD1312 has to recognize the slave address before transmitting or receiving any information by the I²C-bus. The device will respond to the slave address following by the slave address bit ("SA0" bit) and the read/write select bit ("R/W#" bit) with the following byte format,

b ₇	b ₆	b ₅	b ₄	b ₃	b ₂	b ₁	b ₀
0	1	1	1	0	SA0	R/W#	

"SA0" bit provides an extension bit for the slave address. Either "0111100" or "0111101", can be selected as the slave address of SSD1312. D/C# pin acts as SA0 for slave address selection.

"R/W#" bit is used to determine the operation mode of the I²C-bus interface. R/W# = 1, it is in read mode. R/W# = 0, it is in write mode.

- b) I²C-bus data signal (SDA)
SDA acts as a communication channel between the transmitter and the receiver. The data and the acknowledgement are sent through the SDA.

It should be noticed that the ITO track resistance and the pulled-up resistance at "SDA" pin becomes a voltage potential divider. As a result, the acknowledgement would not be possible to attain a valid logic 0 level in "SDA".

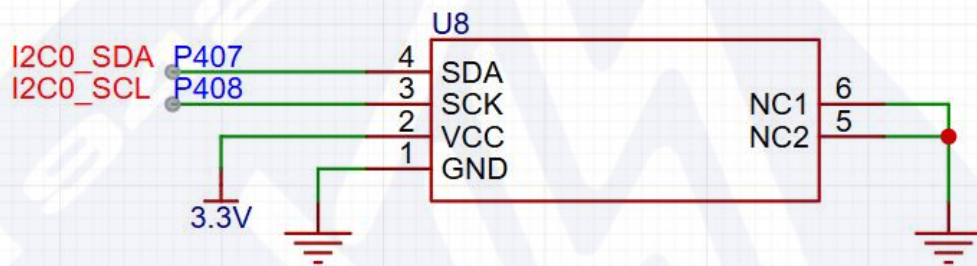
"SDA_{IN}" and "SDA_{OUT}" are tied together and serve as SDA. The "SDA_{IN}" pin must be connected to act as SDA. The "SDA_{OUT}" pin may be disconnected. When "SDA_{OUT}" pin is disconnected, the acknowledgement signal will be ignored in the I²C-bus.

- c) I²C-bus clock signal (SCL)
The transmission of information in the I²C-bus is following a clock signal, SCL. Each transmission of data bit is taken place during a single clock period of SCL.

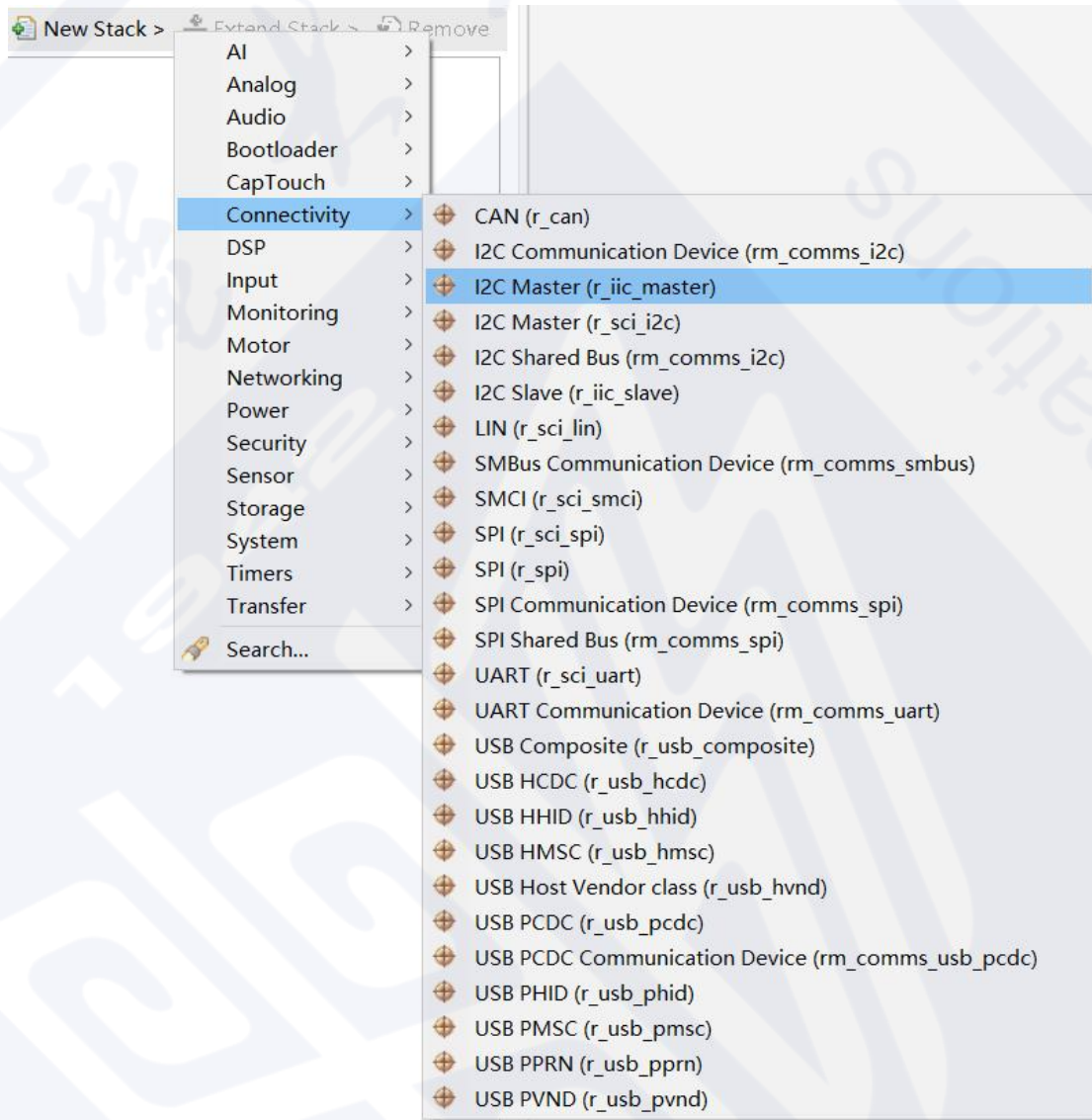
IIC 配置

IIC 接口为 P407 和 P408。

OLED屏幕接口



点击 Stacks->New Stack->Connectivity -> I2C Master(r_sci_master)。

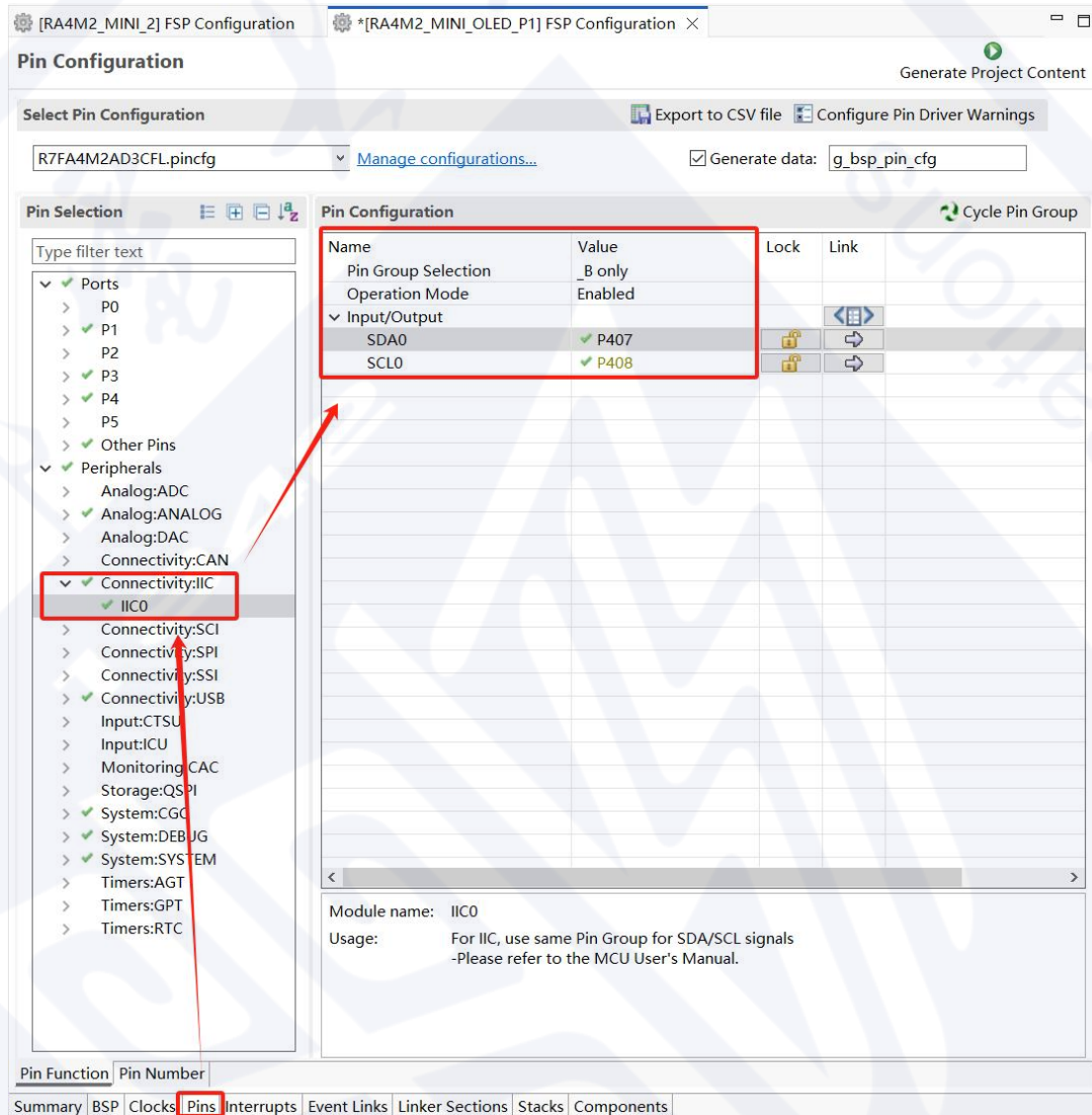


设置 IIC 的配置，需要注意从机的地址，这里从机地址为 0x3c。

- 名称 (Name)：设置为 `g_i2c_master0`，表示这是 I2C 主机的实例名称，用于代码中引用。
- 通道 (Channel)：设为 `0`，表示使用的是 IIC 通道 0。
- 通讯速率模式 (Rate)：设置为 `Fast-mode`，即使用 400kHz 作为 I2C 通信速度。
- 自定义速率 (Custom Rate (bps))：精确设置通信速率为 400000bps。
- 设备地址 (Slave Address)：设为 `0x3C`，表示从设备的 7 位地址，常见于 OLED 屏。
- 地址位数模式 (Address Mode)：设置为 `7-Bit`，即使用标准的 7 位地址。
- 超时模式 (Timeout Mode)：设置为 `Short Mode`，表示启用短时超时机制。
- 回调函数 (Callback)：设置为 `i2c_master_callback`，用于在通信完成或出错时触发处理函数。
- 中断优先级 (Interrupt Priority Level)：设置为 `Priority 12`，用于控制中断响应优先级。
- SDA 管脚 (SDA0)：分配在 P407，引脚用于数据传输线。
- SCL 管脚 (SCL0)：分配在 P408，引脚用于时钟线。

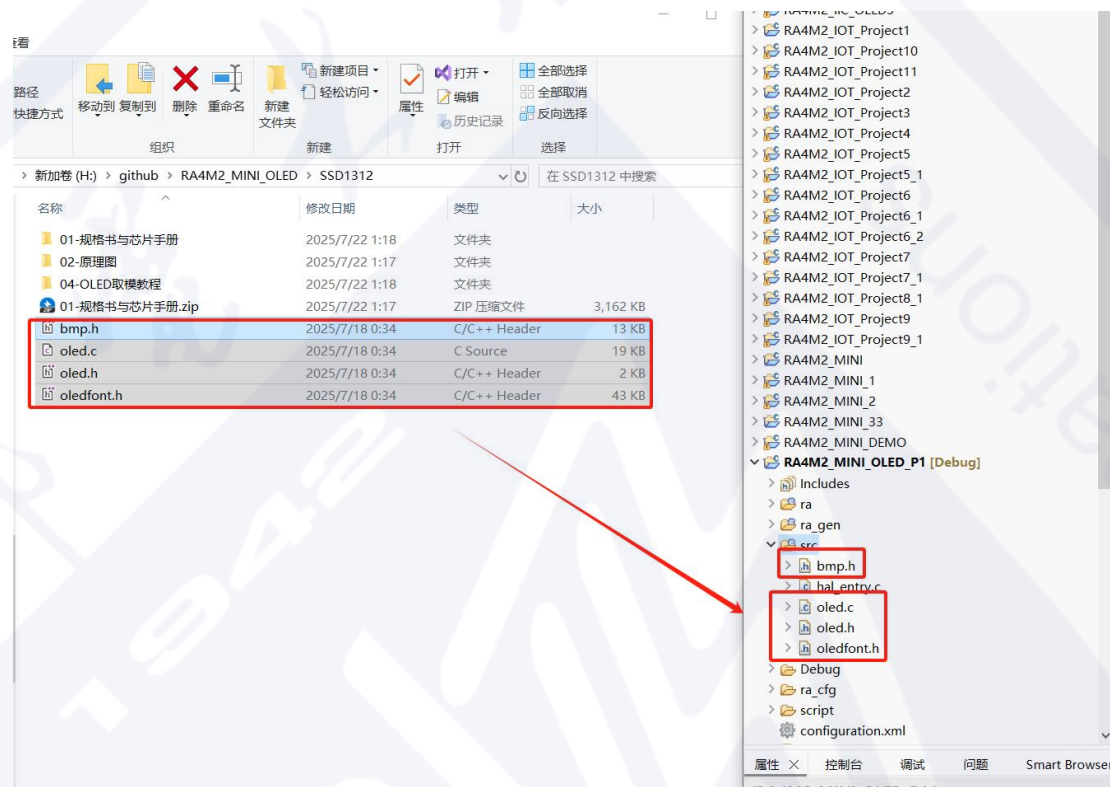
属性 × 控制台 调试 问题 Smart Browser Pin Conflicts 错误日志		
g_i2c_master0 I2C Master (r_iic_master)		
Settings	属性	值
API Info	▼ Common	
	Parameter Checking	Default (BSP)
	DTC on Transmission and Reception	Disabled
	10-bit slave addressing	Disabled
	▼ Module g_i2c_master0 I2C Master (r_iic_master)	
	Name	g_i2c_master0 名称
	Channel	0 通道
	Rate	Fast-mode 通讯速率
	Custom Rate (bps)	400000
	Rise Time (ns)	120
	Fall Time (ns)	120
	Duty Cycle (%)	50
	Slave Address	0x3c 设备地址
	Address Mode	7-Bit 地址位数
	Timeout Mode	Short Mode
	Timeout during SCL Low	Enabled
	Callback	i2c_master_callback 回调函数
	Interrupt Priority Level	Priority 12 优先级
	▼ Pins	
	SDA0	P407
	SCL0	P408 对应管脚

选择对应的 IIC 管脚。



导入驱动

将这四个文件成功加入到 src 源文件目录中。



bmp.h 这是图像数据头文件，包含多个 **BMP[]** 数组，例如 **BMP1[]**、**BMP2[]**、**BMP3[]** 等。

- 作用：保存点阵图像的原始像素数据，通常从 **.bmp** 图转换而来。
- 用途：用于在 **OLED** 上显示图标、**LOGO** 或图形界面。
- 配合函数：使用 **OLED_ShowPicture()** 将这些图片数据渲染到屏幕上。

oled.c 这是 **OLED** 控制逻辑的实现文件，负责初始化和图形绘制等操作。

- 核心功能：
 - **OLED_Init()**：初始化 **OLED** 显示屏。
 - **OLED_WR_Byte()**、**Send_Byte()**：通过 **I2C** 向 **OLED** 发送命令或数据。
 - **OLED_Refresh()**：将显示缓冲区 **OLED_GRAM** 中的数据刷新到屏幕。
 - **OLED_DrawPoint()**、**OLED_DrawLine()**、**OLED_ShowString()** 等：提供点、线、字符、字符串的绘制功能。
 - **OLED_ShowPicture()**：显示 **bmp.h** 中的图片数组。

oled.h 是 **OLED** 驱动函数的头文件，定义了所有对外接口和常量。

- 内容包括：
 - 所有在 **oled.c** 中实现的函数声明。
 - **OLED_CMD** 和 **OLED_DATA** 宏，用于命令/数据标志。
 - 屏幕尺寸相关宏（**oled_x**, **oled_y**）自动适配横屏或竖屏。

- 类型定义: u8, u16, u32。
- 提供清晰的接口给主程序调用, 如 OLED_ShowString()、OLED_Clear() 等。

oledfont.h 这是字体库文件, 用于显示字符和数字时的字模数据。

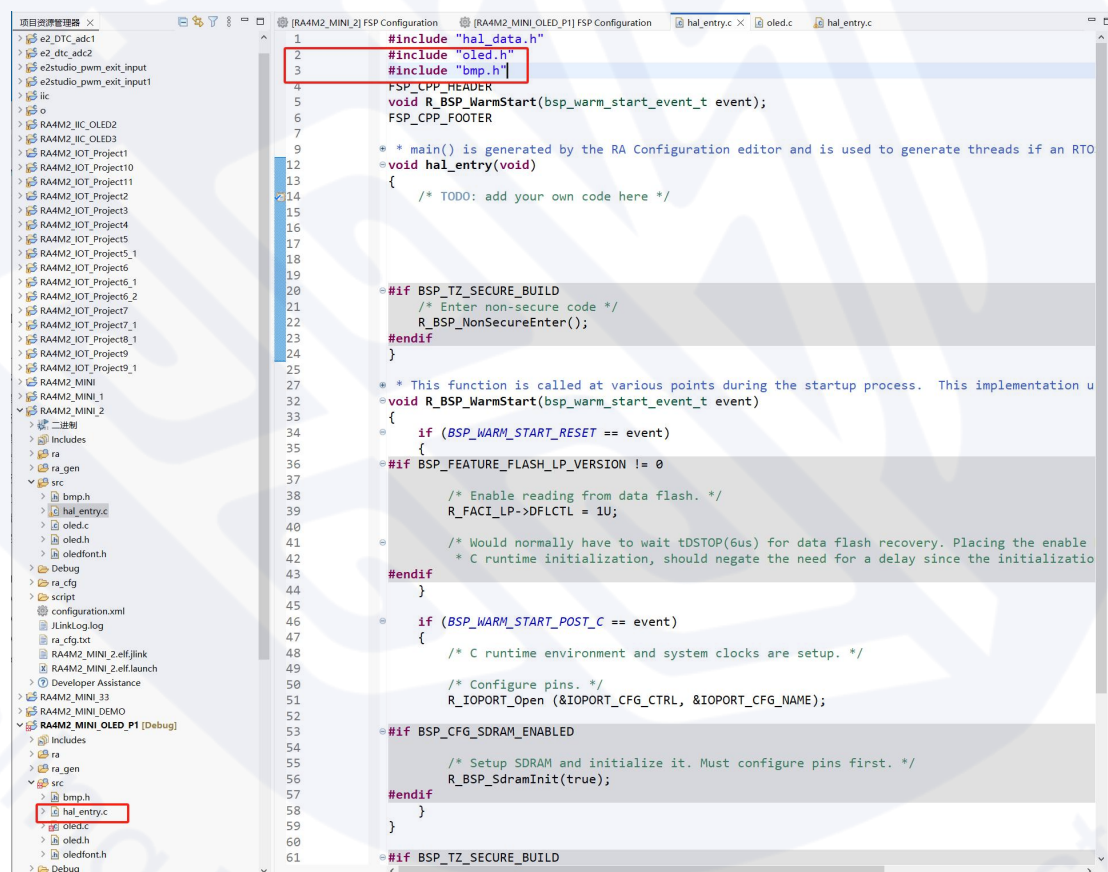
- 内容: 包含 6×8、8×16 等常用 ASCII 字符的点阵定义。
- 用途: 配合 OLED_ShowChar() 或 OLED_ShowString() 函数使用, 按指定位置和大小在 OLED 屏幕上显示文字。

头文件

添加头文件。

```
#include "oled.h"

#include "bmp.h"
```



R_IIC_MASTER_Open() 函数原型

◆ R_IIC_MASTER_Open()

```
fsp_err_t R_IIC_MASTER_Open ( i2c_master_ctrl_t *const p_api_ctrl,
                              i2c_master_cfg_t const *const p_cfg
                              )
```

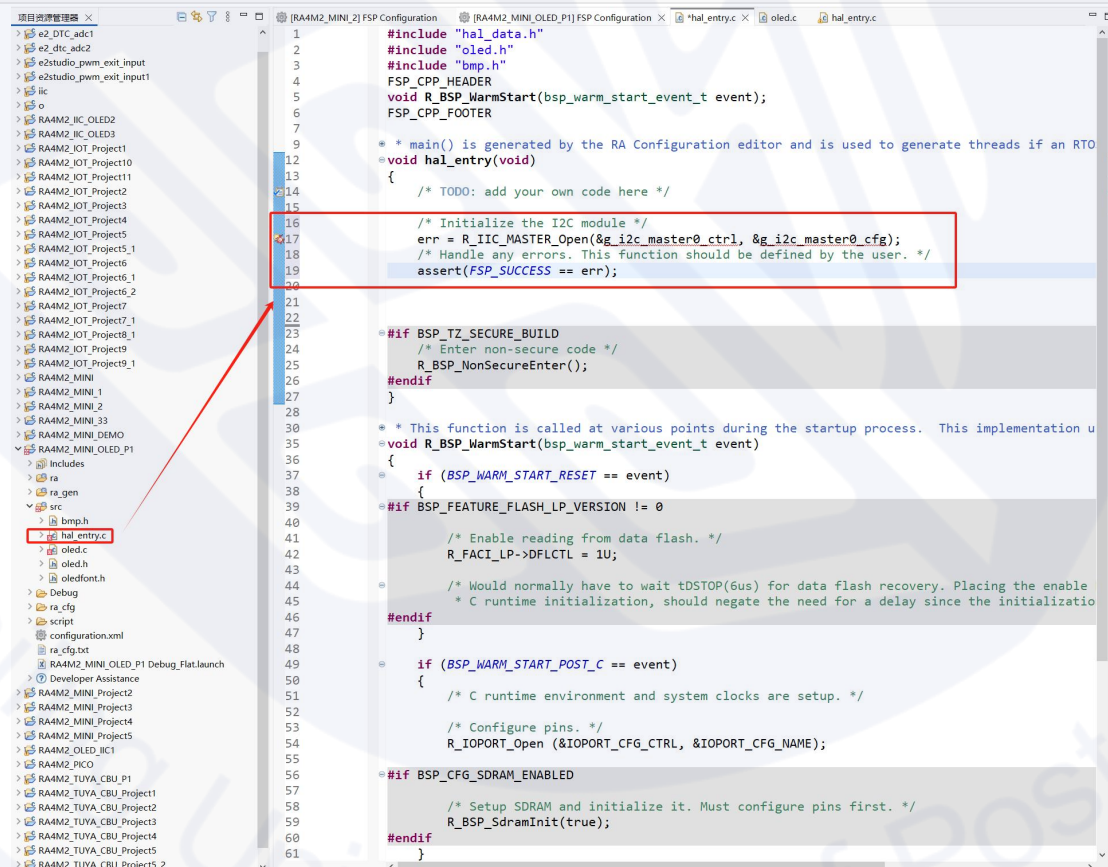
Opens the I2C device.

Return values

FSP_SUCCESS	Requested clock rate was set exactly.
FSP_ERR_ALREADY_OPEN	Module is already open.
FSP_ERR_IP_CHANNEL_NOT_PRESENT	Channel is not available on this MCU.
FSP_ERR_ASSERTION	Parameter check failure due to one or more reasons below: 1. p_api_ctrl or p_cfg is NULL. 2. extended parameter is NULL. 3. Callback parameter is NULL. 4. Set the rate to fast mode plus on a channel which does not support it. 5. Invalid IRQ number assigned

R_IIC_MASTER_Open()函数为执行 IIC 初始化，开启配置如下所示。

```
/* Initialize the I2C module */
err = R_IIC_MASTER_Open(&g_i2c_master0_ctrl, &g_i2c_master0_cfg);
/* Handle any errors. This function should be defined by the user. */
assert(FSP_SUCCESS == err);
```



R_IIC_MASTER_Write()函数原型

◆ R_IIC_MASTER_Write()

```
fsp_err_t R_IIC_MASTER_Write ( i2c_master_ctrl_t *const p_api_ctrl,  
                               uint8_t *const      p_src,  
                               uint32_t const      bytes,  
                               bool const          restart  
                               )
```

Performs a write to the I2C device. The caller will be notified when the operation has completed (successfully) by an I2C_MASTER_EVENT_TX_COMPLETE in the callback.

Return values

FSP_SUCCESS	Function executed without issue.
FSP_ERR_ASSERTION	p_api_ctrl or p_src is NULL.
FSP_ERR_INVALID_SIZE	Provided number of bytes more than uint16_t size (65535) while DTC is used for data transfer.
FSP_ERR_IN_USE	Bus busy condition. Another transfer was in progress.
FSP_ERR_NOT_OPEN	Handle is not initialized. Call R_IIC_MASTER_Open to initialize the control block.

R_IIC_MASTER_Write()函数是向 IIC 设备中写入数据，写入格式如下所示。

```
err = R_IIC_MASTER_Write(&g_i2c_master0_ctrl, &reg, 1, true);  
assert(FSP_SUCCESS == err);
```

Send_Byte(u8 dat) 函数的 I2C 发送实现逻辑，原来用的是模拟 I²C (bit-banging) 方式，但已经注释掉了那段代码，并改用 Renesas FSP 的 I2C Master API 实现硬件 I2C 发送。

```
[RA4M2_MINI_2] FSP Configuration [RA4M2_MINI_OLED_P1] FSP Configuration hal_entry.c oled.c x hal_entry.c
40
41 // 写入一个字节
42 void Send_Byte(u8 dat)
43 {
44     // u8 i;
45     // for(i=0;i<8;i++)
46     // {
47     //     if(dat&0x80)//将dat的8位从最高位依次写入
48     //     {
49     //         OLED_SDA_Set();
50     //     }
51     //     else
52     //     {
53     //         OLED_SDA_Clr();
54     //     }
55     //     IIC_delay();
56     //     OLED_SCL_Set();
57     //     IIC_delay();
58     //     OLED_SCL_Clr();//将时钟信号设置为低电平
59     //     dat<<=1;
60     // }
61
62     R_IIC_MASTER_SlaveAddressSet(&g_i2c_master0_ctrl, 0x3C, I2C_MASTER_ADDR_MODE_7BIT);
63     assert(FSP_SUCCESS == err);
64
65     err = R_IIC_MASTER_Write(&g_i2c_master0_ctrl, &dat, 1, true);
66     assert(FSP_SUCCESS == err);
67     /* Since there is nothing else to do, block until Callback triggers*/
68     //while ((I2C_MASTER_EVENT_TX_COMPLETE != i2c_event) && timeout_ms)
69     while ((I2C_MASTER_EVENT_TX_COMPLETE != i2c_event) && timeout_ms>0)
70     {
71         R_BSP_SoftwareDelay(100U, BSP_DELAY_UNITS_MICROSECONDS);
72         timeout_ms--;
73     }
74     if (I2C_MASTER_EVENT_ABORTED == i2c_event)
75     {
76         __BKPT(0);
77     }
78     /* Read data back from the I2C slave */
79     i2c_event = I2C_MASTER_EVENT_ABORTED;
80     timeout_ms = 100000;
81
82
83
84
85 }
```

OLED_WR_Byte() 函数是 标准的 Renesas FSP I2C OLED 写入函数，它将命令或数据写入 OLED 控制器，区别在于控制字节 0x00 或 0x40，整个过程使用硬件 I2C 保证时序稳定。

```
[RA4M2_MINI_2] FSP Configuration [RA4M2_MINI_OLED_P1] FSP Configuration hal_entry.c oled.c x hal_entry.c
86
87 //发送一个字节
88 //mode: 数据/命令标志 0,表示命令;1,表示数据;
89 void OLED_Wr_Byte(u8 dat,u8 mode)
90 {
91 // I2C_Start();
92 // Send_Byte(0x78);
93 // I2C_WaitAck();
94 // if(mode){Send_Byte(0x40);}
95 // else{Send_Byte(0x00);}
96 // I2C_WaitAck();
97 // Send_Byte(dat);
98 // I2C_WaitAck();
99 // I2C_Stop();
100
101 R_IIC_MASTER_SlaveAddressSet(&g_i2c_master0_ctrl, 0x3C, I2C_MASTER_ADDR_MODE_7BIT);
102 assert(FSP_SUCCESS == err);
103 uint8_t ii[2]={0x00,0x00};
104 if(mode)
105     ii[0] = 0x40;
106 else
107     ii[0] = 0x00;
108 ii[1] = dat;
109
110 err = R_IIC_MASTER_Write(&g_i2c_master0_ctrl, ii, 0x02, true);
111 assert(FSP_SUCCESS == err);
112 /* Since there is nothing else to do, block until Callback triggers*/
113 //while ((I2C_MASTER_EVENT_TX_COMPLETE != i2c_event) && timeout_ms)
114 while ((I2C_MASTER_EVENT_TX_COMPLETE != i2c_event) && timeout_ms>0)
115 {
116     R_BSP_SoftwareDelay(100U, BSP_DELAY_UNITS_MICROSECONDS);
117     timeout_ms--;
118 }
119 if (I2C_MASTER_EVENT_ABORTED == i2c_event)
120 {
121     __BKPT(0);
122 }
123 /* Read data back from the I2C slave */
124 i2c_event = I2C_MASTER_EVENT_ABORTED;
125 timeout_ms = 100000;
126
127
128
129
130 }
```

R_IIC_MASTER_Read() 函数原型

◆ R_IIC_MASTER_Read()

```
fsp_err_t R_IIC_MASTER_Read ( i2c_master_ctrl_t *const p_api_ctrl,
                               uint8_t *const p_dest,
                               uint32_t const bytes,
                               bool const restart
                             )
```

Performs a read from the I2C device. The caller will be notified when the operation has completed (successfully) by an I2C_MASTER_EVENT_RX_COMPLETE in the callback.

Return values

FSP_SUCCESS	Function executed without issue.
FSP_ERR_ASSERTION	p_api_ctrl, p_dest or bytes is NULL.
FSP_ERR_INVALID_SIZE	Provided number of bytes more than uint16_t size (65535) while DTC is used for data transfer.
FSP_ERR_IN_USE	Bus busy condition. Another transfer was in progress.
FSP_ERR_NOT_OPEN	Handle is not initialized. Call R_IIC_MASTER_Open to initialize the control block.

R_IIC_MASTER_Read()函数是向 IIC 设备中读取数据，读取格式如下所示。

```
/* Read data from I2C slave */
err = R_IIC_MASTER_Read(&g_i2c_master0_ctrl, bufp, len, false);
assert(FSP_SUCCESS == err);
```

i2c_master_callback() 回调函数

对于数据是否发送完毕，可以查看是否获取到 I2C_MASTER_EVENT_TX_COMPLETE 字段。

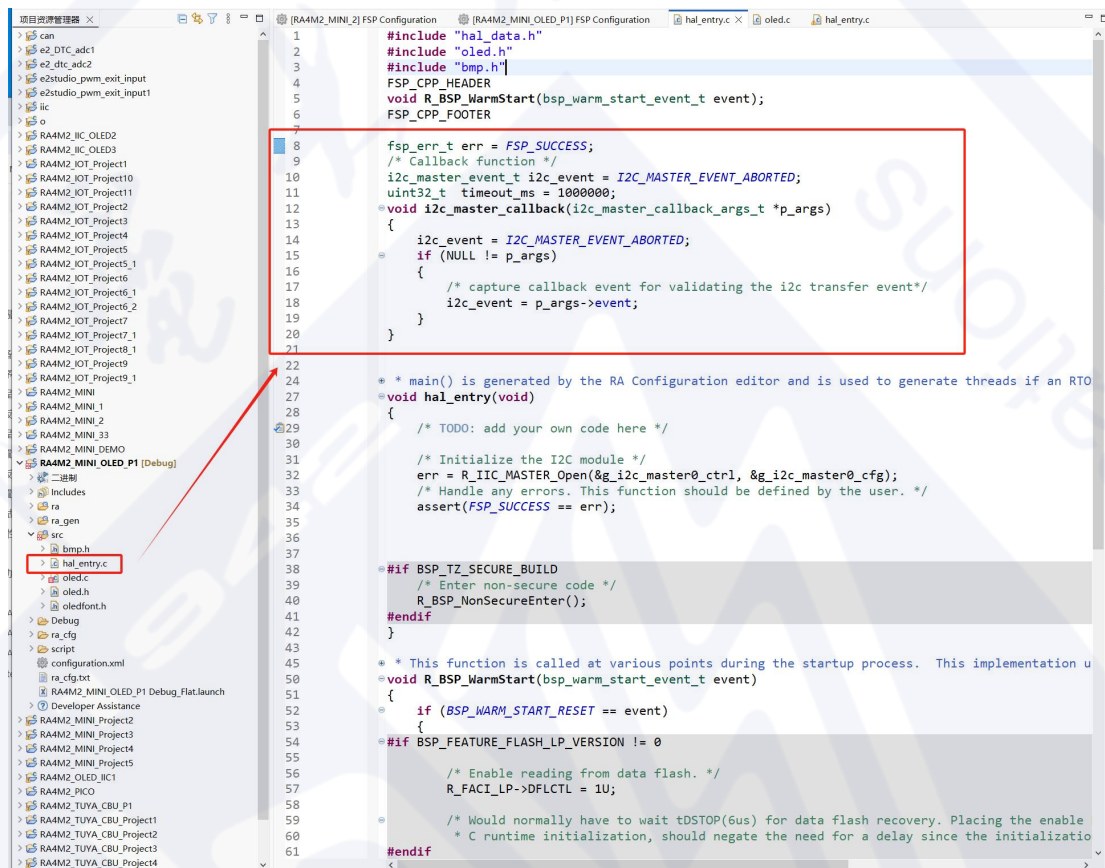
◆ i2c_master_event_t

enum i2c_master_event_t

Callback events

Enumerator	
I2C_MASTER_EVENT_ABORTED	A transfer was aborted.
I2C_MASTER_EVENT_RX_COMPLETE	A receive operation was completed successfully.
I2C_MASTER_EVENT_TX_COMPLETE	A transmit operation was completed successfully.

```
fsp_err_t err = FSP_SUCCESS;
/* Callback function */
i2c_master_event_t i2c_event = I2C_MASTER_EVENT_ABORTED;
uint32_t timeout_ms = 1000000;
void i2c_master_callback(i2c_master_callback_args_t *p_args)
{
    i2c_event = I2C_MASTER_EVENT_ABORTED;
    if (NULL != p_args)
    {
        /* capture callback event for validating the i2c transfer event*/
        i2c_event = p_args->event;
    }
}
```



初始化屏幕

1. OLED_Init()

- 初始化 OLED 屏幕工作状态。
- 发送一系列命令以配置页寻址模式、电荷泵开启、对比度调节、显示使能等。
- 必须在任何显示函数调用之前执行。

2. OLED_Clear()

- 清空 OLED 显存缓冲区 OLED_GRAM。
- 设置为全 0，表示关闭所有像素。
- 并通过 OLED_Refresh() 将清空的显存数据刷新到屏幕，确保视觉上是全黑状态。

3. OLED_ColorTurn(0)

- 设置显示颜色模式。
 - 0 为正显（背景黑，文字白）。
 - 1 为反显（背景白，文字黑）。

- 实际通过向 OLED 控制器写入 0xA6（正显）或 0xA7（反显）命令实现。

4. OLED_DisplayTurn(0)

- 设置显示方向。
 - 0 表示正常方向；
 - 1 表示上下翻转显示。
- 对应命令：
 - 正常显示：0xC8 + 0xA1
 - 翻转显示：0xC0 + 0xA0

5. OLED_ShowString(84, 0, "RA4M2", 16, 1)

- 在指定坐标位置（x=84, y=0）显示字符串 "RA4M2"。
- 字体大小为 16（对应 8x16 字模）。
- 第五个参数为显示模式：1 表示正显。
- 字模数据来源于 oledfont.h，函数内部会将对应点阵写入显存缓冲 OLED_GRAM。

6. OLED_Refresh()

- 显存与实际 OLED 屏幕是分离的。
- 此函数将 OLED_GRAM 中的缓冲数据逐页写入 OLED 控制器，刷新显示内容。
- 若不调用本函数，ShowString() 写入的数据不会真正显示在屏幕上。

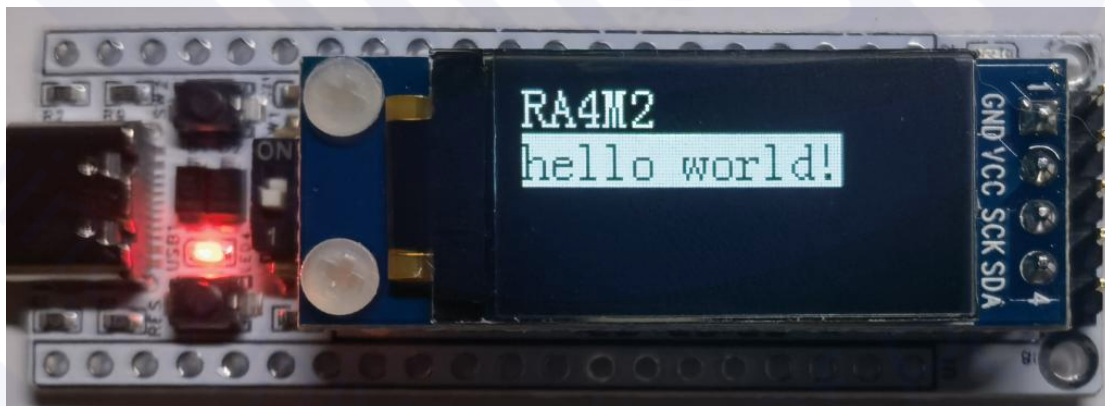
```
/* Initialize the I2C module */
err = R_IIC_MASTER_Open(&g_i2c_master0_ctrl, &g_i2c_master0_cfg);
/* Handle any errors. This function should be defined by the user. */
assert(FSP_SUCCESS == err);

OLED_Init();           // 初始化 OLED 屏幕（发送初始化命令序列，设置工作模式）
OLED_Clear();          // 清空显存（OLED_GRAM），并刷新，使屏幕全黑
OLED_ColorTurn(0);     // 设置显示颜色模式：0 为正常显示，1 为反色显示（黑白反转）
OLED_DisplayTurn(0);   // 设置显示方向：0 为正常方向，1 为上下翻转显示

OLED_Clear();//清空 OLED 显存
OLED_ShowString(0, 0, "RA4M2", 16, 1);//在坐标 (0,0) 显示 RA4M2，字体 16，正显
OLED_ShowString(0, 16, "hello world!", 16, 0);//在坐标 (0,16) 显示 hello world!，字体 16，反显
OLED_Refresh();// 将 OLED_GRAM 显存内容刷新到 OLED 屏幕上，显示数字和字符串

R_BSP_SoftwareDelay(2000, BSP_DELAY_UNITS_MILLISECONDS); // 延时 200 毫秒
```

```
*hal_entry.c x startup.c main.c bsp_common.c
3      #include "bmp.h"
4      FSP_CPP_HEADER
5      void R_BSP_WarmStart(bsp_warm_start_event_t event);
6      FSP_CPP_FOOTER
7
8      fsp_err_t err = FSP_SUCCESS;
9      /* Callback function */
10     i2c_master_event_t i2c_event = I2C_MASTER_EVENT_ABORTED;
11     uint32_t timeout_ms = 1000000;
12     void i2c_master_callback(i2c_master_callback_args_t *p_args)
13     {
14         i2c_event = I2C_MASTER_EVENT_ABORTED;
15         if (NULL != p_args)
16         {
17             /* capture callback event for validating the i2c transfer event*/
18             i2c_event = p_args->event;
19         }
20     }
21
22
23
24     /* * main() is generated by the RA Configuration editor and is used to generate threads if an RT
25     void hal_entry(void)
26     {
27         /* TODO: add your own code here */
28
29         /* Initialize the I2C module */
30         err = R_IIC_MASTER_Open(&g_i2c_master0_ctrl, &g_i2c_master0_cfg);
31         /* Handle any errors. This function should be defined by the user. */
32         assert(FSP_SUCCESS == err);
33
34         OLED_Init();           // 初始化 OLED 屏幕 (发送初始化命令序列, 设置工作模式)
35         OLED_Clear();          // 清空显存 (OLED_GRAM), 并刷新, 使屏幕全黑
36         OLED_ColorTurn(0);     // 设置显示颜色模式: 0 为正常显示, 1 为反色显示 (黑白反转)
37         OLED_DisplayTurn(0);   // 设置显示方向: 0 为正常方向, 1 为上下翻转显示
38
39         OLED_Clear(); // 清空 OLED 显存
40         /* pointer targets in passing argument 3 of 'OLED_ShowString' differ in signedness [-Wpointer
41         OLED_ShowString(0, 0, "RA4M2", 16, 1); // 在坐标 (84,0) 显示RA4M2, 字体16, 正显
42
43         OLED_Refresh(); // 将 OLED_GRAM 显存内容刷新到 OLED 屏幕上, 显示数字和字符串
44
45         R_BSP_SoftwareDelay(2000, BSP_DELAY_UNITS_MILLISECONDS); // 延时 200 毫秒
46
47
48
49
50
51
52     #if BSP_TZ_SECURE_BUILD
53         /* Enter non-secure code */
54         R_BSP_NonSecureEnter();
55     #endif
56 }
```

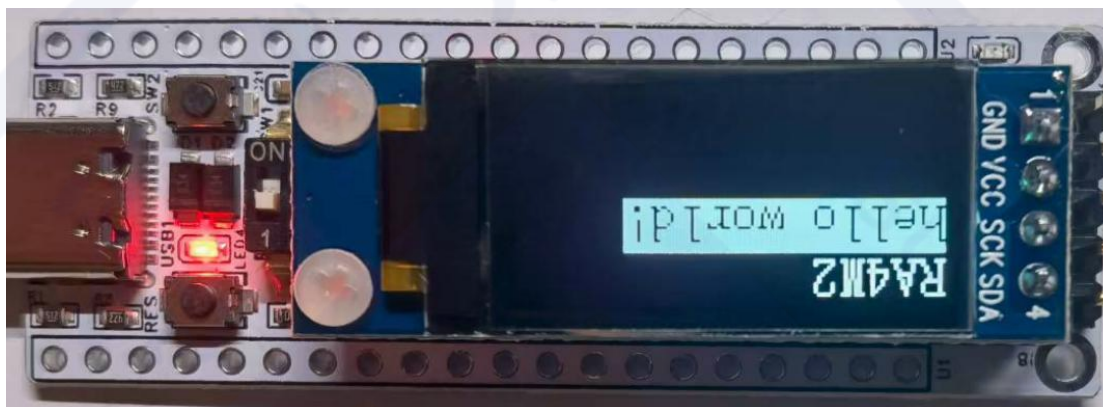


OLED 显示方向

OLED_DisplayTurn(uint8_t i) 是用于设置 OLED 显示方向（上下翻转）的函数。

控制 OLED 屏幕是 正常方向显示 还是 上下翻转显示，通常用于设备安装方向不同、或旋转 UI 的情况。

```
*hal_entry.c x startup.c main.c bsp_common.c oled.c
2      #include "oled.h"
3      #include "bmp.h"
4      FSP_CPP_HEADER
5      void R_BSP_WarmStart(bsp_warm_start_event_t event);
6      FSP_CPP_FOOTER
7
8      fsp_err_t err = FSP_SUCCESS;
9      /* Callback function */
10     i2c_master_event_t i2c_event = I2C_MASTER_EVENT_ABORTED;
11     uint32_t timeout_ms = 1000000;
12     void i2c_master_callback(i2c_master_callback_args_t *p_args)
13     {
14         i2c_event = I2C_MASTER_EVENT_ABORTED;
15         if (NULL != p_args)
16         {
17             /* capture callback event for validating the i2c transfer event*/
18             i2c_event = p_args->event;
19         }
20     }
21
22
23
24     /* * main() is generated by the RA Configuration editor and is used to generate threads if an RT
25
26
27     void hal_entry(void)
28     {
29         /* TODO: add your own code here */
30
31         /* Initialize the I2C module */
32         err = R_IIC_MASTER_Open(&g_i2c_master0_ctrl, &g_i2c_master0_cfg);
33         /* Handle any errors. This function should be defined by the user. */
34         assert(FSP_SUCCESS == err);
35
36         OLED_Init();           // 初始化 OLED 屏幕（发送初始化命令序列，设置工作模式）
37         OLED_Clear();          // 清空显存（OLED_GRAM），并刷新，使屏幕全黑
38         OLED_ColorTurn(0);     // 设置显示颜色模式：0 为正常显示，1 为反色显示（黑白反转）
39         OLED_DisplayTurn(1);   // 设置显示方向：0 为正常方向，1 为上下翻转显示
40
41         OLED_Clear();          // 清空 OLED 显存
42         OLED_ShowString(0, 0, "RA4M2", 16, 1); // 在坐标 (84,0) 显示RA4M2, 字体16, 正显
43         OLED_ShowString(0, 16, "hello world!", 16, 0); // 在坐标 (0,16) 显示hello world!, 字体16, 反显
44         OLED_Refresh();        // 将 OLED_GRAM 显存内容刷新到 OLED 屏幕上，显示数字和字符串
45
46         R_BSP_SoftwareDelay(2000, BSP_DELAY_UNITS_MILLISECONDS); // 延时 200 毫秒
47
48
49
50
51
52     #if BSP_TZ_SECURE_BUILD
53         /* Enter non-secure code */
54         R_BSP_NonSecureEnter();
55     #endif
56 }
57
```



project2: 点阵字库与图形

概述

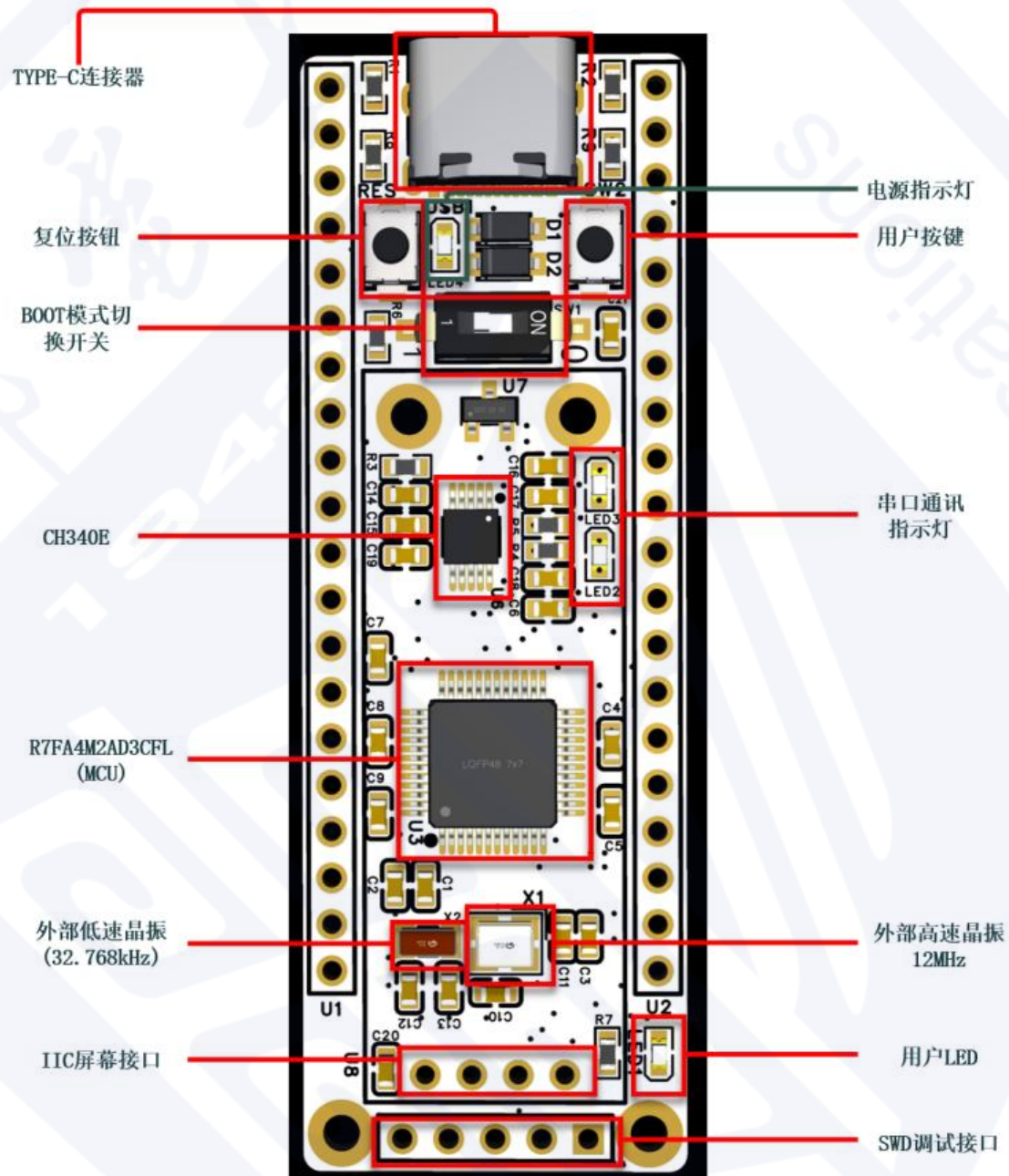
本章节将介绍如何在 SSD1312 OLED 屏幕上实现中文字符和图像的显示。相较于英文字母，中文由于字符数量庞大、点阵复杂，需要使用外部字库或自定义字模来支持显示。本讲内容将围绕中文字符的点阵原理、自建简易字模库、显示函数实现方式等进行详细解析。同时，本章节也将讲解如何通过点阵方式绘制图片(如 LOGO 或 ICON)，并展示如何将 BMP 图像转换为适配 OLED 的点阵数据，实现图形显示功能。通过本讲的学习，开发者将掌握 OLED 图文混排的核心技巧，为界面美化与功能展示打下基础。



硬件准备

首先需要准备一个开发板，这里我准备的是自己绘制的开发板，需要的可以进行申请。

主控为 R7FA4M2AD3CFL



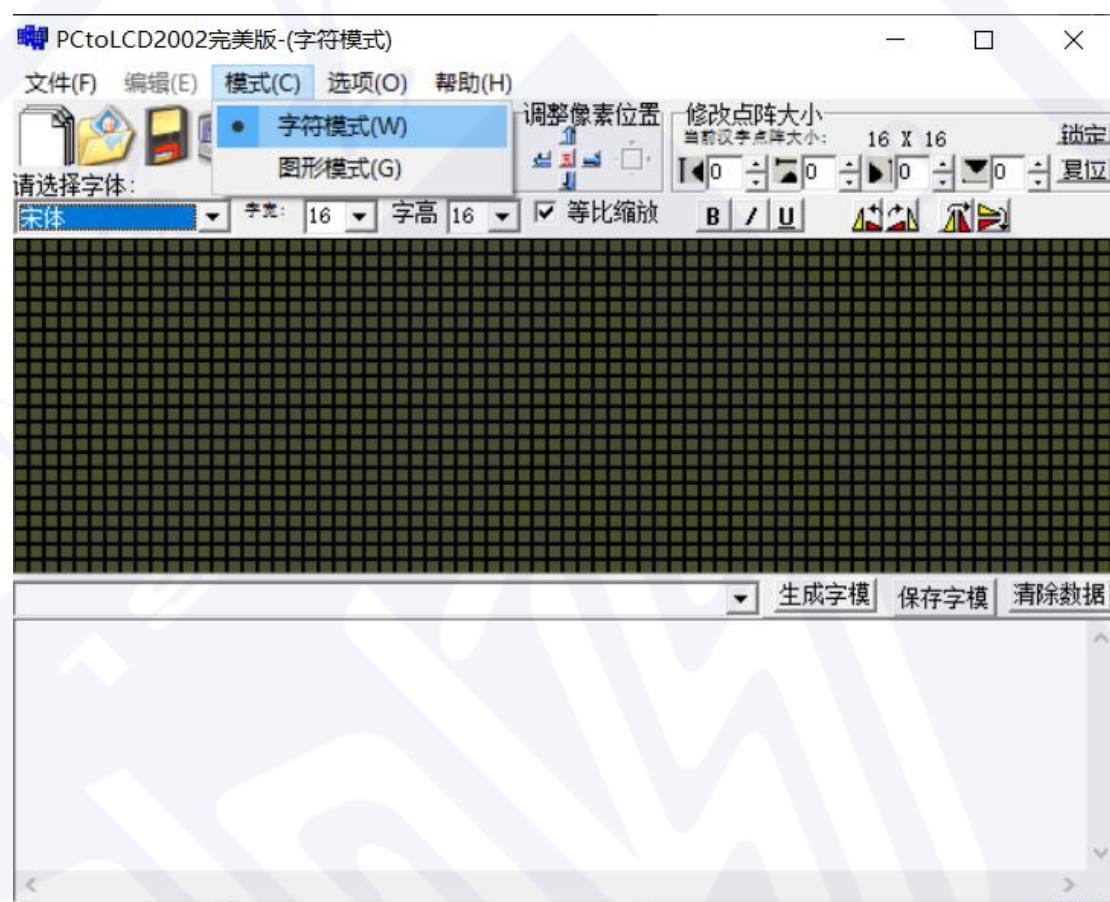
参考程序

https://github.com/CoreMaker-lab/RA4M2_MINI_OLED

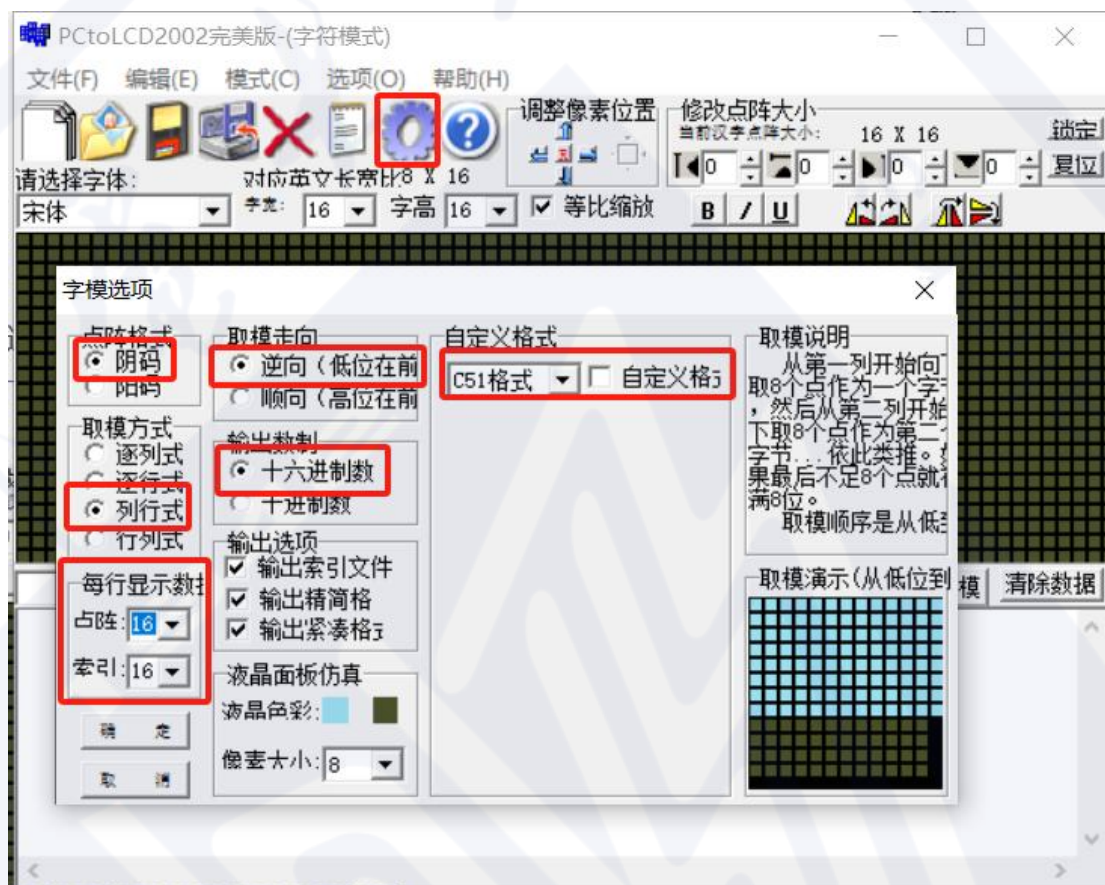
https://gitee.com/CoreMaker/RA4M2_MINI_OLED

字符取模

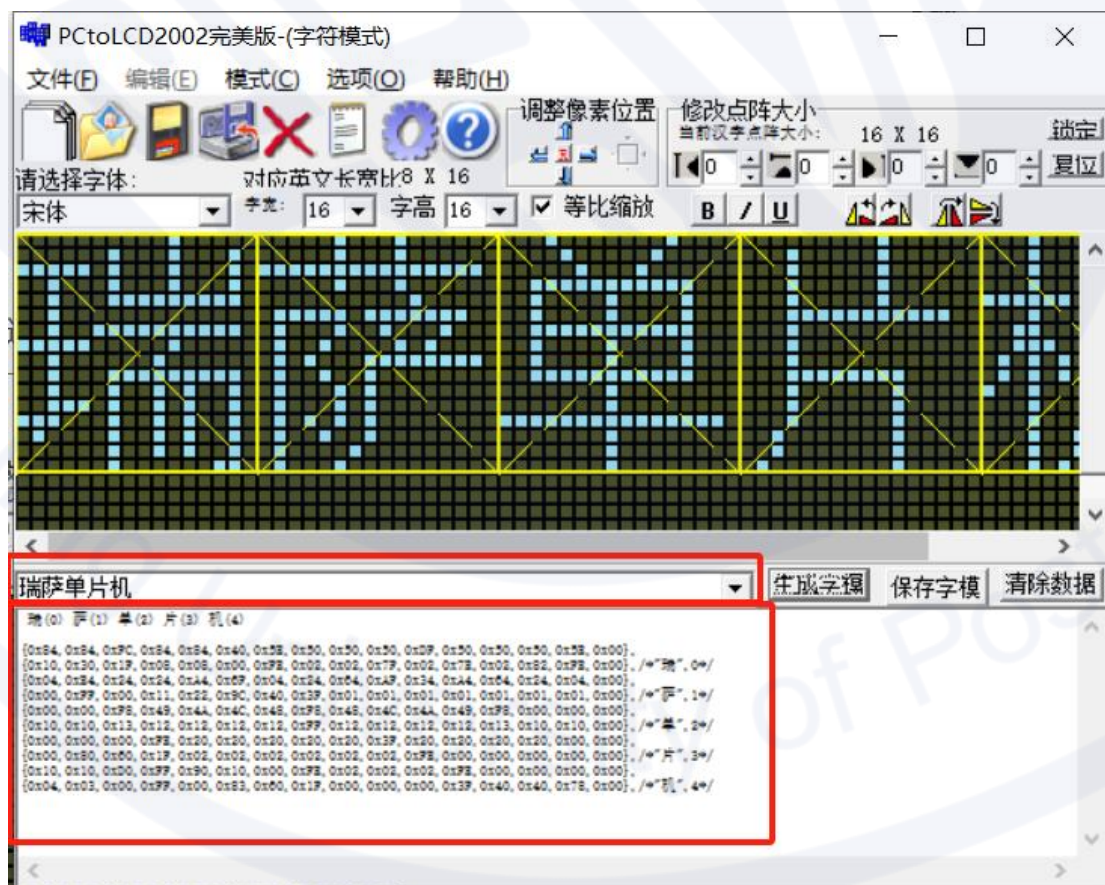
由于单片机的 flash 比较吃紧，所以要显示汉字需要先进行取模。首先需要设置为字符模式。



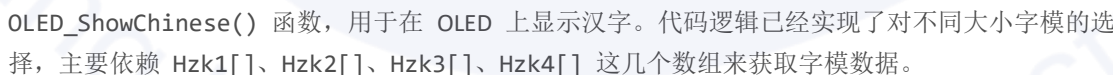
取模软件设置。



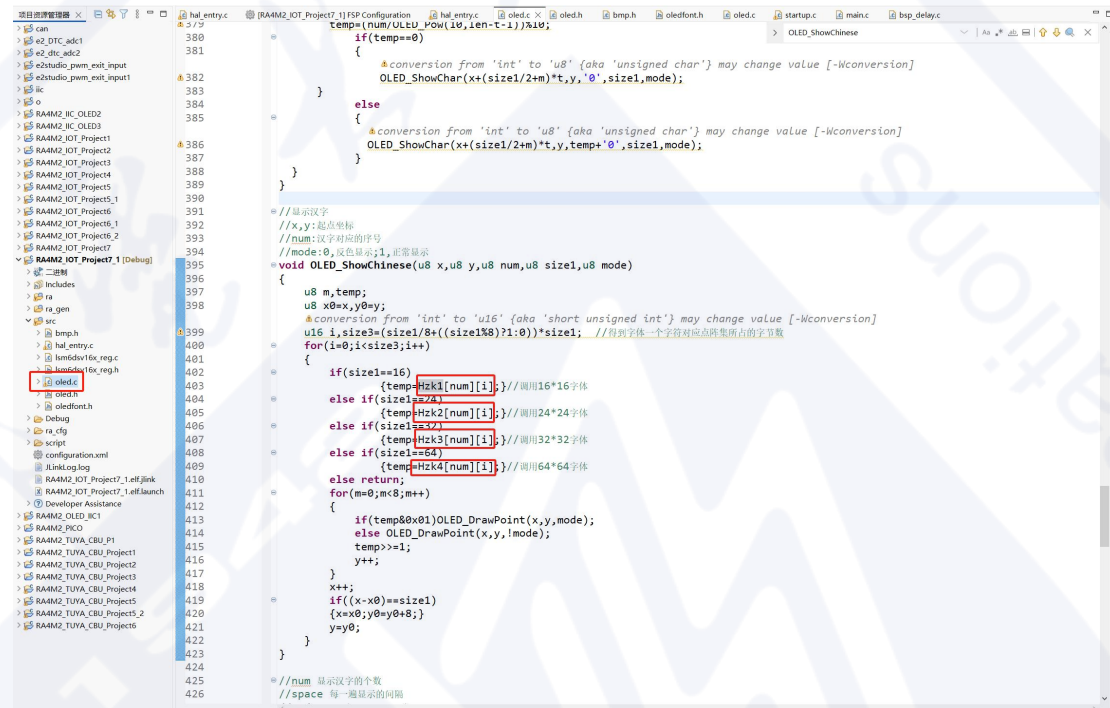
设置完之后就可以取自己想要汉字的编码，输入完毕点击生成字模即可。



```
const unsigned char Hzkl[][32]={
{0x84,0x84,0xFC,0x84,0x84,0x40,0x5E,0x50,0x50,0x50,0xDF,0x50,0x50,0x50,0x5E,0x00,0x10,0x30,0
x1F,0x08,0x08,0x00,0xFE,0x02,0x02,0x7F,0x02,0x7E,0x02,0x82,0xFE,0x00},/*"瑞",0*/
{0x04,0xE4,0x24,0x24,0xA4,0x6F,0x04,0x24,0x64,0xAF,0x34,0xA4,0x64,0x24,0x04,0x00,0x00,0xFF,0
x00,0x11,0x22,0x9C,0x40,0x3F,0x01,0x01,0x01,0x01,0x01,0x01,0x01,0x00},/*"萨",1*/
{0x00,0x00,0xF8,0x49,0x4A,0x4C,0x48,0xF8,0x48,0x4C,0x4A,0x49,0xF8,0x00,0x00,0x00,0x10,0x10,0
x13,0x12,0x12,0x12,0x12,0xFF,0x12,0x12,0x12,0x12,0x13,0x10,0x10,0x00},/*"单",2*/
{0x00,0x00,0x00,0xFE,0x20,0x20,0x20,0x20,0x20,0x3F,0x20,0x20,0x20,0x20,0x00,0x00,0x00,0x80,0
x60,0x1F,0x02,0x02,0x02,0x02,0x02,0x02,0xFE,0x00,0x00,0x00,0x00,0x00},/*"片",3*/
{0x10,0x10,0xD0,0xFF,0x90,0x10,0x00,0xFE,0x02,0x02,0x02,0xFE,0x00,0x00,0x00,0x00,0x04,0x03,0
x00,0xFF,0x00,0x83,0x60,0x1F,0x00,0x00,0x00,0x3F,0x40,0x40,0x78,0x00},/*"机",4*/
};
```



OLED_ShowChinese() 函数, 用于在 OLED 上显示汉字。代码逻辑已经实现了对不同大小字模的选择, 主要依赖 Hzk1[], Hzk2[], Hzk3[], Hzk4[] 这几个数组来获取字模数据。



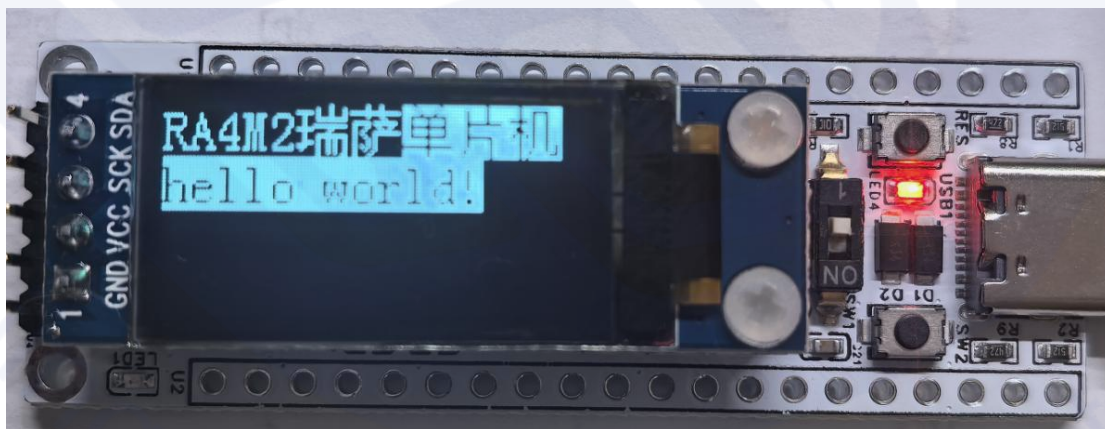
在 OLED_Start 添加汉字显示驱动。

```

OLED_ShowChinese(40,0,0,16,1); //在坐标 (40,0) 显示 Hzk1[0], 即汉字“瑞”, 正显
OLED_ShowChinese(56,0,1,16,1); //在坐标 (56,0) 显示 Hzk1[1], 即汉字“萨”, 正显
OLED_ShowChinese(72,0,2,16,0); //在坐标 (72,0) 显示 Hzk1[2], 即汉字“单”, 反色
OLED_ShowChinese(88,0,3,16,0); //在坐标 (88,0) 显示 Hzk1[3], 即汉字“片”, 反色
OLED_ShowChinese(104,0,4,16,0); //在坐标 (104,0) 显示 Hzk1[4], 即汉字“机”, 反色
OLED_Refresh(); // 将 OLED_GRAM 显存内容刷新到 OLED 屏幕上, 显示数字和字符串

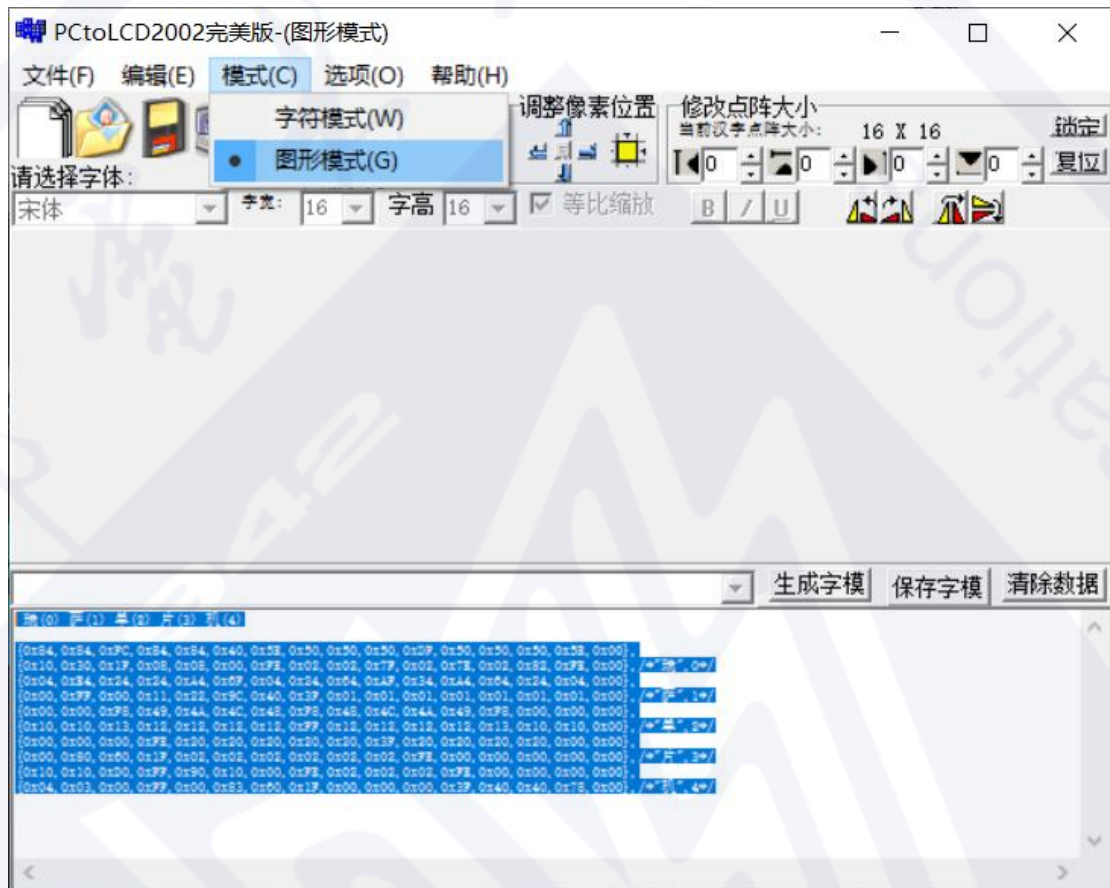
```

```
[RA4M2_MINI_OLED_P1] FSP Configuration  *hal_entry.c x oledfont.h startup.c main.c
16 {
17     /* capture callback event for validating the i2c transfer event*/
18     i2c_event = p_args->event;
19 }
20
21
22
23
24 * * main() is generated by the RA Configuration editor and is used to generate threads if an RTOS is used. T
25 void hal_entry(void)
26 {
27     /* TODO: add your own code here */
28
29     /* Initialize the I2C module */
30     err = R_IIC_MASTER_Open(&g_i2c_master0_ctrl1, &g_i2c_master0_cfg);
31     /* Handle any errors. This function should be defined by the user. */
32     assert(FSP_SUCCESS == err);
33
34     OLED_Init();          // 初始化 OLED 屏幕 (发送初始化命令序列, 设置工作模式)
35     OLED_Clear();         // 清空显存 (OLED_GRAM), 并刷新, 使屏幕全黑
36     OLED_ColorTurn(0);    // 设置显示颜色模式; 0 为正常显示, 1 为反色显示 (黑白反转)
37     OLED_DisplayTurn(1);  // 设置显示方向: 0 为正常方向, 1 为上下翻转显示
38
39     OLED_Clear(); // 清空 OLED 显存
40     /* pointer targets in passing argument 3 of 'OLED_ShowString' differ in signedness [-Wpointer-sign]
41     OLED_ShowString(0, 0, "RA4M2", 16, 1); // 在坐标 (84,0) 显示RA4M2, 字体16, 正显
42     /* pointer targets in passing argument 3 of 'OLED_ShowString' differ in signedness [-Wpointer-sign]
43     OLED_ShowString(0, 16, "hello world!", 16, 0); // 在坐标 (0,16) 显示hello world!, 字体16, 反显
44     OLED_Refresh(); // 将 OLED_GRAM 显存内容刷新到 OLED 屏幕上, 显示数字和字符串
45
46     R_BSP_SoftwareDelay(2000, BSP_DELAY_UNITS_MILLISECONDS); // 延时 200 毫秒
47
48     OLED_ShowChinese(40,0,0,16,1); // 在坐标 (40,0) 显示 Hzk1[0], 即汉字“瑞”, 正显
49     OLED_ShowChinese(56,0,1,16,1); // 在坐标 (56,0) 显示 Hzk1[1], 即汉字“萨”, 正显
50     OLED_ShowChinese(72,0,2,16,0); // 在坐标 (72,0) 显示 Hzk1[2], 即汉字“单”, 反色
51     OLED_ShowChinese(88,0,3,16,0); // 在坐标 (88,0) 显示 Hzk1[3], 即汉字“片”, 反色
52     OLED_ShowChinese(104,0,4,16,0); // 在坐标 (104,0) 显示 Hzk1[4], 即汉字“机”, 反色
53     OLED_Refresh(); // 将 OLED_GRAM 显存内容刷新到 OLED 屏幕上, 显示数字和字符串
54
55
56
57 #if BSP_TZ_SECURE_BUILD
58     /* Enter non-secure code */
59     R_BSP_NonSecureEnter();
60 #endif
61 }
62
63 * * This function is called at various points during the startup process. This implementation uses the event
64 void R_BSP_WarmStart(bsp_warm_start_event_t event)
65 {
66     if (BSP_WARM_START_RESET == event)
67     {
68         #if BSP_FEATURE_FLASH_LP_VERSION != 0
69
70             /* Enable reading from data flash. */
71
72
73
74
75
```

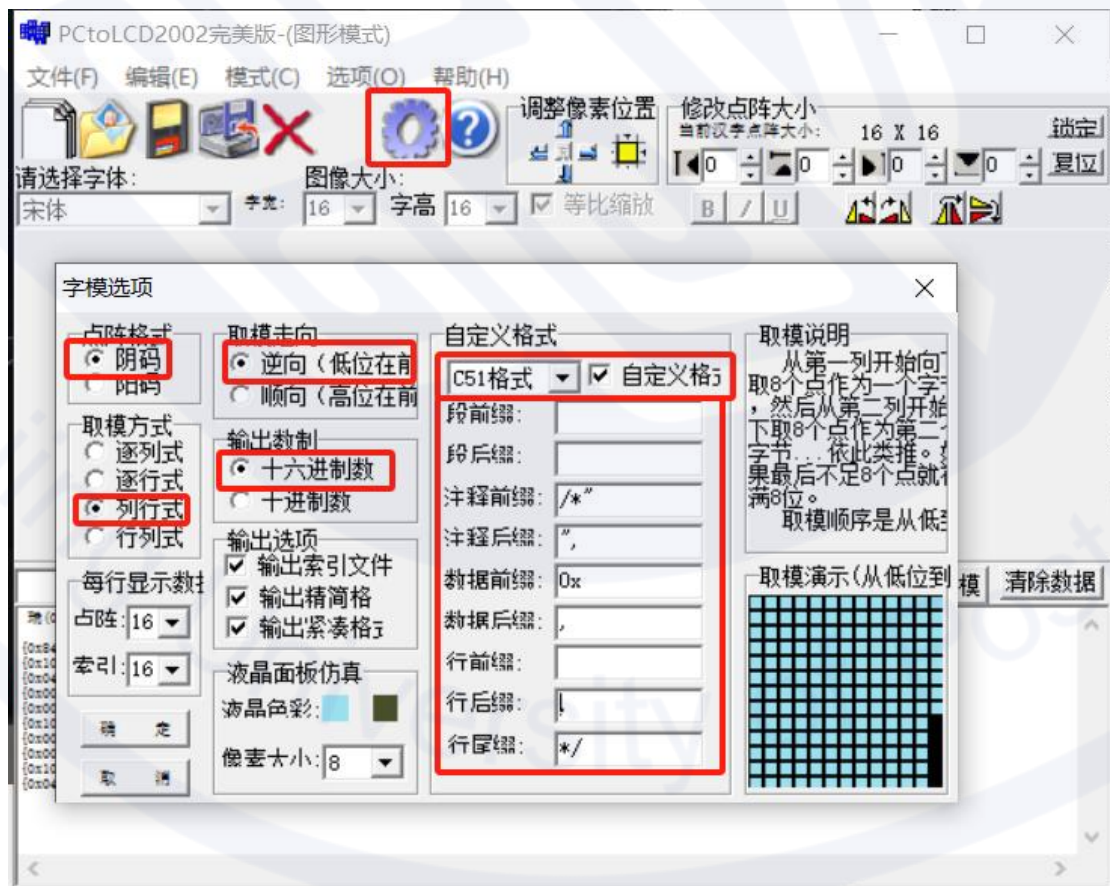


图片取模

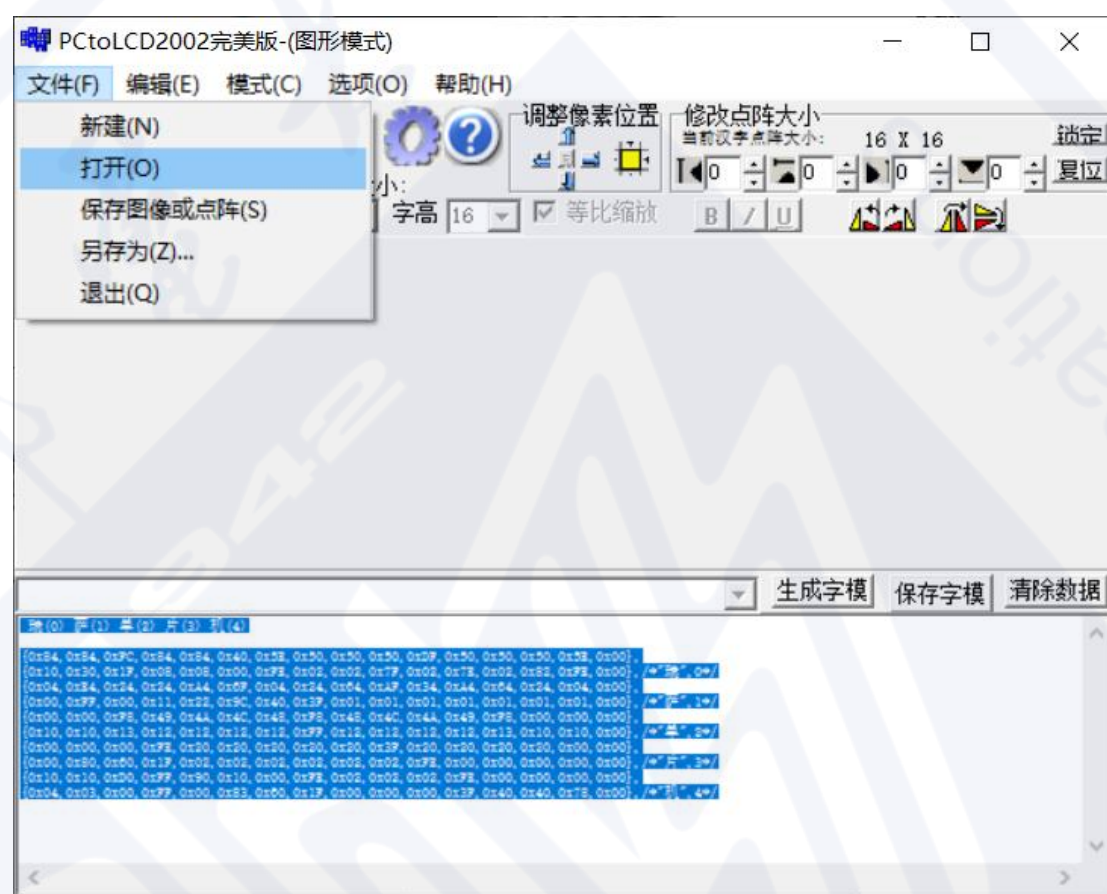
设置为图形模式。



格式设置。

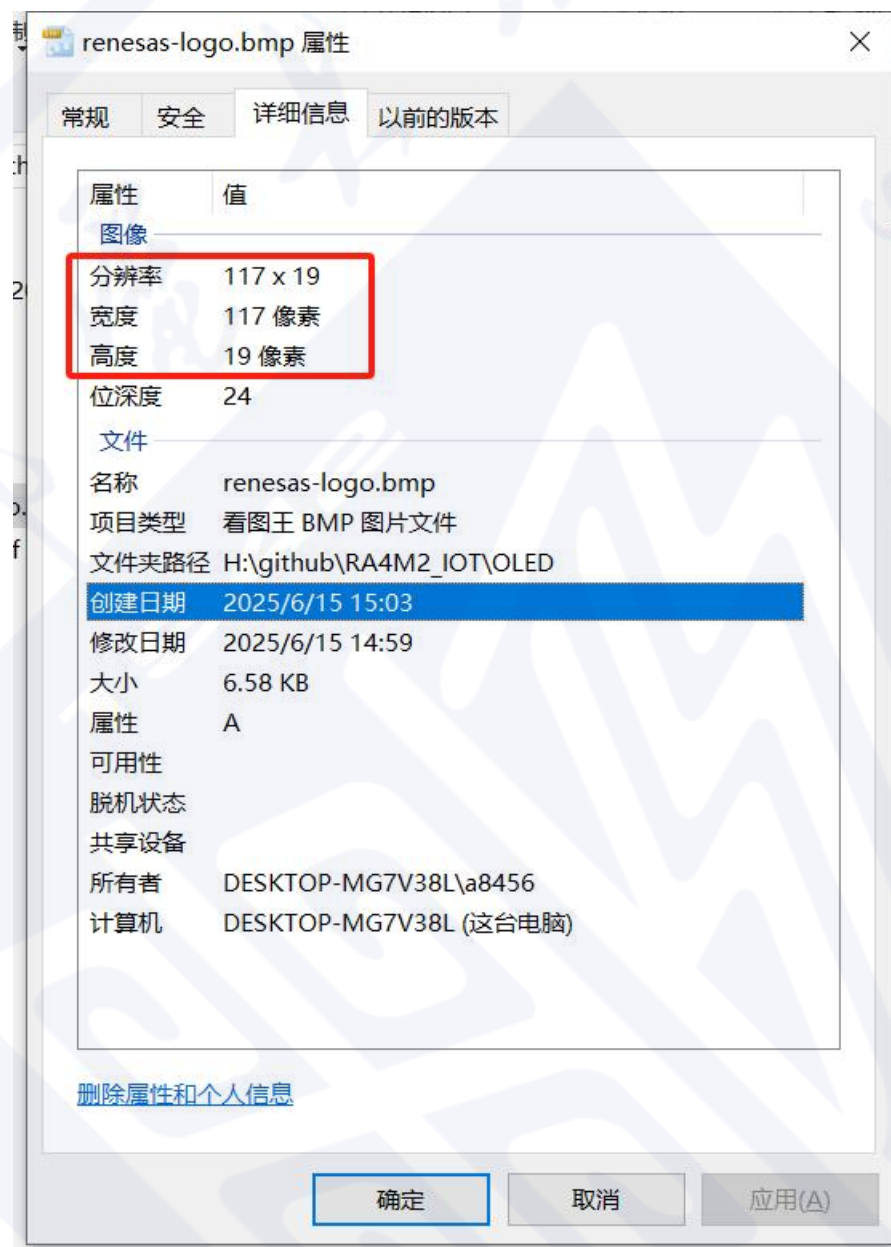


选择需要打开的文件。



打开的文件必须为 bmp 文件。


```
unsigned char BMP2[] =
{
0x1E,0x0E,0x07,0x07,0x07,0x07,0x03,0x03,0x83,0x83,0xC7,0xC7,0xFF,0xFF,0xFF,0xFF,
0xFF,0x7E,0x3C,0xE0,0xF0,0xF8,0xF8,0xF8,0xF8,0xBC,0xBC,0xBC,0xBC,0xBC,0xBC,
0xBC,0xBC,0x1C,0xFC,0xFC,0xFC,0xFC,0xF8,0xF8,0xF8,0xF8,0xE0,0xC0,0x80,0x00,0x00,
0xFC,0xFC,0xFC,0xFC,0x00,0xE0,0xF0,0xF8,0xF8,0xF8,0xF8,0xBC,0xBC,0xBC,0xBC,
0xBC,0xBC,0xBC,0x9C,0xE0,0xF0,0xF8,0xF8,0xF8,0xF8,0xFC,0xFC,0xFC,0xBC,0xBC,0xBC,
0x3C,0x3C,0x3C,0x3C,0x1C,0x00,0x00,0x80,0xC0,0xF0,0xF0,0xF8,0xF8,0xFC,0xFC,0xFC,
0xF8,0xF8,0xC0,0x80,0x00,0xF0,0xF0,0xF8,0xF8,0xF8,0xF8,0xFC,0xFC,0xBC,0xBC,0xBC,
0xBC,0x3C,0x3C,0x3C,0x1C,0xFF,0xFF,0xFF,0xFF,0xFF,0xFF,0xFF,0x00,0x03,0x07,0x0F,
0x1F,0x3F,0x7F,0xFF,0xFD,0xF8,0xF0,0xE0,0xFF,0xBF,0x7F,0xFF,0xFF,0xFF,0xF7,0xE7,
0xE7,0xE7,0xE7,0xE7,0xE7,0xE3,0xE0,0xFF,0xFF,0xFF,0xFF,0xFF,0x03,0x07,0x0F,
0x3F,0x3F,0x7F,0xFF,0xFE,0xFF,0xFF,0xFF,0xFF,0x00,0x3F,0x7F,0xFF,0xFF,0xFF,0xFF,
0xF7,0xE7,0xE7,0xE7,0xE7,0xE7,0xE3,0xE3,0xE0,0xE0,0xE1,0xE3,0xE3,0xE7,0xE7,
0xEF,0xEF,0xEF,0xFF,0xFF,0xFF,0xFF,0xFE,0x7E,0xFC,0xFC,0xFE,0xFF,0xFF,0x7F,0x1F,
0xEF,0xE1,0xE1,0xE3,0xEF,0xFF,0xFF,0xFF,0xFF,0xFE,0xFD,0xF1,0xE3,0xE3,0xE7,0xE7,
0xE7,0xEF,0xEF,0xEF,0xFF,0xFF,0xFF,0xFF,0xFE,0x7E,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x01,0x01,0x03,0x03,0x07,0x07,
0x07,0x06,0x04,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,/*"H:\github\RA4M
2_MINI_OLED\renesas-logo.bmp",0*/
};
```

在 OLED 屏幕下半部分显示一张 117×19 的图片，并刷新屏幕显示。

```
OLED_ShowPicture(0, 32, 117, 19, BMP2, 1);           // 在 OLED 屏幕的 (x=0, y=32) 位置显示一张宽 117、高 19 像素的图片 BMP3，正常模式显示

OLED_Refresh();                                     // 将图片内容从缓冲区刷新到 OLED 屏幕上，真正显示出来

R_BSP_SoftwareDelay(200, BSP_DELAY_UNITS_MILLISECONDS); // 延时 200 毫秒
```

```

16      {
17          /* capture callback event for validating the i2c transfer event*/
18          i2c_event = p_args->event;
19      }
20  }
21
22
23
24  /* * main() is generated by the RA Configuration editor and is used to generate threads if an RTOS is used. T
25  void hal_entry(void)
26  {
27      /* TODO: add your own code here */
28
29      /* Initialize the I2C module */
30      err = R_IIC_MASTER_Open(&g_i2c_master0_ctrl, &g_i2c_master0_cfg);
31      /* Handle any errors. This function should be defined by the user. */
32      assert(FSP_SUCCESS == err);
33
34      OLED_Init();          // 初始化 OLED 屏幕 (发送初始化命令序列, 设置工作模式)
35      OLED_Clear();         // 清空显存 (OLED_GRAM), 并刷新, 使屏幕全黑
36      OLED_ColorTurn(0);    // 设置显示颜色模式: 0 为正常显示, 1 为反色显示 (黑白反转)
37      OLED_DisplayTurn(1);  // 设置显示方向: 0 为正常方向, 1 为上下翻转显示
38
39      OLED_Clear(); // 清空 OLED 显存
40      /*pointer targets in passing argument 3 of 'OLED_ShowString' differ in signedness [-Wpointer-sign]
41      OLED_ShowString(0, 0, "RA4M2", 16, 1); // 在坐标 (84,0) 显示 RA4M2, 字体16, 正显
42      /*pointer targets in passing argument 3 of 'OLED_ShowString' differ in signedness [-Wpointer-sign]
43      OLED_ShowString(0, 16, "hello world!", 16, 0); // 在坐标 (0,16) 显示hello world!, 字体16, 反显
44      OLED_Refresh(); // 将 OLED_GRAM 显存内容刷新到 OLED 屏幕上, 显示数字和字符串
45
46      R_BSP_SoftwareDelay(2000, BSP_DELAY_UNITS_MILLISECONDS); // 延时 2000 毫秒
47
48      OLED_ShowChinese(40,0,0,16,1); // 在坐标 (40,0) 显示 Hzk1[0], 即汉字“瑞”, 正显
49      OLED_ShowChinese(56,0,1,16,1); // 在坐标 (56,0) 显示 Hzk1[1], 即汉字“萨”, 正显
50      OLED_ShowChinese(72,0,2,16,0); // 在坐标 (72,0) 显示 Hzk1[2], 即汉字“单”, 反色
51      OLED_ShowChinese(88,0,3,16,0); // 在坐标 (88,0) 显示 Hzk1[3], 即汉字“片”, 反色
52      OLED_ShowChinese(104,0,4,16,0); // 在坐标 (104,0) 显示 Hzk1[4], 即汉字“机”, 反色
53      OLED_Refresh(); // 将 OLED_GRAM 显存内容刷新到 OLED 屏幕上, 显示数字和字符串
54
55      OLED_ShowPicture(0, 32, 117, 19, BMP2, 1); // 在 OLED 屏幕的 (x=0, y=32) 位置显示一张宽 117、高
56      OLED_Refresh(); // 将图片内容从缓冲区刷新到 OLED 屏幕上, 真正显示出来
57      R_BSP_SoftwareDelay(200, BSP_DELAY_UNITS_MILLISECONDS); // 延时 200 毫秒
58
59      #if BSP_TZ_SECURE_BUILD
60          /* Enter non-secure code */
61          R_BSP_NonSecureEnter();
62      #endif
63  }
64
65  /* * This function is called at various points during the startup process. This implementation uses the event
66  void R_BSP_WarmStart(bsp_warm_start_event_t event)
67  {
68      if (BSP_WARM_START_RESET == event)
69      {
70          #if BSP_FEATURE_FLASH_IP_VERSION != 0
71

```

